

Ausarbeitung Projekt

Data Science Virtueller Tutor Elektronik-Labor

Joannis Kopp (1350403)

Philip Lesche (1682942)

Jannis Fink (1684087)

16. August 2026

Contents

1	Einleitung	4
2	Grundlagen	6
2.1	Sokratische Didaktik und Tutoring	6
2.2	Retrieval-Augmented Generation	7
2.2.1	Sparse und Dense Retrieval	7
2.2.2	Hybrid Retrieval und Reciprocal Rank Fusion	8
2.2.3	Cross-Encoder-Reranking	8
2.3	Evaluation LLM-basierter Dialogsysteme	9
2.3.1	LLM-as-a-Judge	10
2.3.2	Multi-Turn-Evaluation und ConversationalTestCase	10
2.3.3	Conversational G-Eval	11
2.3.4	Vordefinierte Multi-Turn-Metriken	11
3	Untersuchungsgegenstand	13
3.1	Die RAG-Pipeline	13
3.2	Die Prompt-Varianten für RQ1	15
3.3	RAG-Konfigurationen für RQ2	16
4	Methodik	18
4.1	Untersuchungsdesign	18
4.2	Untersuchungsgegenstand und Datengrundlage	18
4.3	Evaluationsarchitektur	19
4.4	Verwendete Methoden und Werkzeuge	19
4.5	Bewertungskriterien	20
4.5.1	Sokratische Qualität (RQ1)	20
4.5.2	Retrieval-Qualität (RQ2)	21
4.6	Durchführung	22
4.7	Gütekriterien und methodische Einschränkungen	23
4.8	Projekt-Setup	23
5	Ergebnisse	24
5.1	RQ1 – Vergleich der Prompt-Varianten	24
5.1.1	Ergebnisse je Kriterium	24
5.1.2	Ergebnisstabilität und Streuung	26
5.1.3	Gewichteter Gesamtscore und Gate-Compliance	27
5.1.4	Charakterisierung der einzelnen Varianten	27
5.2	RQ2 – Vergleich der RAG-Konfigurationen	28
5.2.1	Datenlage und Retrieval-Abdeckung	28

5.2.2	Kontextbezogene Metriken und effektive Scores	30
5.3	Querschnitt: Verhalten, Thema und Niveau	31
5.3.1	Verteilung der Einzelscores	31
5.3.2	Ergebnisse nach Studentenverhalten	32
5.3.3	Ergebnisse nach Thema und Niveau	33
5.3.4	Qualitative Analyse der Unter-Schwellen-Fälle	33
6	Diskussion	36
6.1	Beantwortung von RQ1 und RQ2	36
6.2	Metrik Mismatch (Goal Accuracy und Completeness vs sokratisches Ziel) und methodische Reflexion	36
6.3	Datenqualität (leere Gespräche und 30 % Retrieval Abdeckung) und deren Einfluss	37
6.4	Limitationen (Stichprobengröße, Chunk Granularität, Judge Bias und Thresh- old Wahl)	38
	References	39
A	Anhang	40
A.1	Verwendete Prompt-Varianten	40
A.2	Baseline ohne Systemprompt	40
A.3	Minimaler sokratischer Prompt	42

1 Einleitung

Large Language Models ermöglichen die Umsetzung dialogbasierter Tutorsysteme, deren Verhalten über Instruktionen in natürlicher Sprache gesteuert und durch zusätzliche Wissensquellen erweitert werden kann. Für den Einsatz als virtueller Tutor im Elektroniklabor reicht es jedoch nicht aus, fachlich korrekte Antworten zu erzeugen. Der Tutor soll Studierende bei der eigenständigen Auseinandersetzung mit einer Problemstellung unterstützen, anstatt ihnen unmittelbar eine fertige Lösung vorzugeben. Gleichzeitig muss der für die Antwortgenerierung bereitgestellte fachliche Kontext zur jeweiligen Fragestellung passen und von dem Sprachmodell angemessen genutzt werden.

Die vorliegende Arbeit untersucht einen LLM-basierten virtuellen Tutor für das Elektroniklabor, der ein sokratisches Tutorverhalten mit einer Retrieval-Augmented-Generation-Pipeline verbindet. Das Tutorverhalten wird dabei durch einen Systemprompt gesteuert, während die RAG-Pipeline fachliche Inhalte aus Vorlesungs- und Labormaterialien abrufen und dem Sprachmodell als zusätzlichen Kontext bereitstellt. Die resultierende Qualität des Systems hängt damit sowohl von der Ausgestaltung der Instruktionen als auch von der Konfiguration des Retrievals ab.

Ziel der Untersuchung ist es, diese beiden Einflussfaktoren getrennt voneinander zu evaluieren. Daraus ergeben sich zwei Forschungsfragen:

- RQ1:** Welchen Einfluss hat die Ausgestaltung des Systemprompts auf die sokratische Qualität des Tutorverhaltens?
- RQ2:** Welchen Einfluss hat die Konfiguration der RAG-Pipeline auf die Qualität des bereitgestellten Retrieval-Kontexts und dessen Nutzung bei der Antwortgenerierung?

Zur Beantwortung von RQ1 werden vier Prompt-Varianten mit unterschiedlichem Grad der didaktischen Steuerung miteinander verglichen. Diese reichen von einer Baseline ohne sokratische Instruktion über einen minimalen sokratischen Prompt bis zu einem strukturierten sokratischen Prompt und einem domänenspezifischen In-Lab-Tutorprompt. Die RAG-Konfiguration wird innerhalb dieser Auswertungslinie konstant gehalten, sodass die beobachteten Unterschiede auf die Gestaltung des Systemprompts zurückgeführt werden können.

RQ2 untersucht dagegen unterschiedliche Konfigurationen der RAG-Pipeline bei konstant gehaltenem Systemprompt. Variiert werden die verwendete Wissensbasis, das Retrieval-Verfahren sowie der Einsatz eines Cross-Encoder-Rerankings. Betrachtet werden Dense, Sparse und Hybrid Retrieval mit Vorlesungsinhalten, Laborinhalten oder deren Kombination. Ergänzend dient eine Variante ohne Retrieval als Referenz.

Die Evaluation erfolgt anhand synthetisch erzeugter Multi-Turn-Gespräche zwischen einer simulierten studierenden Person und dem virtuellen Tutor. Die Testszenarien unterscheiden sich hinsichtlich des Themengebiets, des Kompetenzniveaus und des simulierten

Studierendenverhaltens. Die resultierenden Gespräche werden anschließend automatisiert mit DeepEval bewertet. Für RQ1 werden anwendungsspezifische Kriterien der sokratischen Qualität sowie die Rollentreue des Tutors betrachtet. RQ2 wird anhand der Relevanz des abgerufenen Kontexts, der Übereinstimmung der Tutorantworten mit diesem Kontext und der Retrieval-Abdeckung untersucht. Die beiden Forschungsfragen werden in getrennten Auswertungslinien betrachtet, sodass jeweils nur der zu untersuchende Faktor variiert wird.

Die Arbeit ist wie folgt aufgebaut: Kapitel 2 legt die theoretischen Grundlagen des sokratischen Tutorings, der Retrieval-Augmented Generation und der Evaluation LLM-basierter Dialogsysteme dar. Anschließend werden der untersuchte Tutor, die RAG-Pipeline sowie die Prompt- und Retrieval-Varianten beschrieben. Das Methodikkapitel erläutert das Untersuchungsdesign, die Datengrundlage, die Evaluationsarchitektur und die verwendeten Bewertungskriterien. Darauf aufbauend werden die Ergebnisse der beiden Forschungsfragen sowie ergänzende Querschnittsanalysen dargestellt. Abschließend werden die Befunde hinsichtlich der Forschungsfragen eingeordnet und die methodischen Einschränkungen der Evaluation diskutiert.

2 Grundlagen

2.1 Sokratische Didaktik und Tutoring

Die sokratische Methode bezeichnet einen Lehransatz, bei dem Erkenntnis nicht primär durch die Vermittlung fertiger Antworten, sondern durch die eigenständige Auseinandersetzung des Lernenden mit Fragen, Annahmen und Begründungen entwickelt werden soll. Eine für die moderne didaktische Rezeption maßgebliche Ausarbeitung geht auf Leonard Nelson zurück. In seinem 1922 gehaltenen und 1929 veröffentlichten Vortrag *Die sokratische Methode* grenzt Nelson sokratischen Unterricht von einer auf Wissensvermittlung ausgerichteten Lehrform ab. Sein Ziel besteht nicht darin, den Lernenden philosophische Ergebnisse zu übermitteln, sondern sie zum eigenständigen Philosophieren anzuleiten [16]. Erkenntnis soll demnach aus Urteilen hervorgehen, die der Lernende selbst fällt und anschließend auf ihre Voraussetzungen zurückführt. Sokratischer Unterricht wird damit als Unterricht im Selbstdenken verstanden [16].

Diese Zielsetzung bestimmt zugleich die Rolle des Lehrenden. Nelson unterscheidet zwischen einer äußeren Anregung des Denkens und der Übernahme eines fremden Urteils. Während die Anregung des Erkenntnisprozesses erforderlich sein kann, soll der Lehrende vermeiden, das Urteil des Lernenden durch die Vorgabe eigener Lösungen zu ersetzen [16]. Die sokratische Methode zielt somit nicht auf das Stellen möglichst vieler Fragen, sondern auf einen Dialog, der die eigenständige Prüfung des eigenen Denkens ermöglicht.

Eine frühe Übertragung der sokratischen Methode auf computergestütztes Tutoring findet sich bei Stevens und Collins [19]. Mit dem *Why System* beschreiben sie einen sokratischen Tutor, dessen Dialogverhalten durch regelbasierte Tutorstrategien gesteuert wird. Das System fordert Lernende unter anderem zu Vorhersagen und Begründungen auf, greift ihre Antworten in Folgefragen auf und setzt Gegenbeispiele ein, um Annahmen zu überprüfen [19]. Dabei soll die Auswahl der Tutorinterventionen nicht unabhängig voneinander erfolgen, sondern sich an übergeordneten Lernzielen und am erkennbaren Wissensstand des Lernenden orientieren [19]. Die Arbeit zeigt damit, wie sich die sokratische Grundidee des angeleiteten eigenständigen Denkens in ein dialogorientiertes Tutorssystem übertragen lässt.

Eine für die weitere Ausgestaltung eines sokratischen Tutors relevante Konkretisierung liefert Gentry [12]. Er grenzt sokratisches Fragen von der bloßen Abfrage faktischen Wissens ab und verbindet es mit dem Ziel, Lernende zum Nachdenken und zur Entwicklung kritischen Denkens anzuregen [12]. Für lernförderliches Fragen formuliert er fünf Leitlinien: Der Tutor soll erstens den Kenntnisstand des Lernenden berücksichtigen und neues Wissen schrittweise auf vorhandenem Wissen aufbauen. Zweitens sollen Fragen einem Lernzweck dienen und nicht um ihrer selbst willen gestellt werden. Drittens soll das Ziel der Fragen für den Lernenden erkennbar sein. Viertens sollen die für das Lernziel maßgeblichen Inhalte in den Mittelpunkt gestellt werden. Fünftens soll die Gesprächsführung

darauf verzichten, Lernende durch Fragen bloßzustellen oder zu demütigen [12].

Für die vorliegende Arbeit wird sokratisches Tutoring entsprechend als dialogische Lehrform verstanden, bei der Fragen nicht der reinen Wissensabfrage dienen, sondern den Lernenden bei der eigenständigen Auseinandersetzung mit einer Problemstellung unterstützen. Der Tutor orientiert seine Fragen am bisherigen Gesprächsverlauf und am erkennbaren Kenntnisstand und führt den Lernenden schrittweise durch den Erkenntnisprozess. Die von Gentry formulierten Leitlinien bilden damit konkrete Anforderungen an das Verhalten des in dieser Arbeit entwickelten LLM-basierten Tutors und werden im weiteren Verlauf bei dessen Evaluation berücksichtigt.

2.2 Retrieval-Augmented Generation

Large Language Models (LLMs) greifen bei der Textgenerierung zunächst auf Wissen zurück, das während des Trainings in ihren Parametern abgebildet wurde. Dieses parametrische Wissen lässt sich jedoch nicht ohne Weiteres aktualisieren oder um anwendungsspezifische Informationen ergänzen. Lewis et al. verbinden deshalb bei der *Retrieval-Augmented Generation* (RAG) ein parametrisches Sprachmodell mit einem externen, nichtparametrischen Wissensspeicher [14]. Vor der Generierung werden für eine Eingabe relevante Inhalte aus diesem Wissensspeicher abgerufen und dem Sprachmodell als zusätzlicher Kontext bereitgestellt. Die erzeugte Antwort kann dadurch neben dem parametrischen Wissen des Modells auf zur Laufzeit abgerufene Informationen zurückgreifen [14]. Ein RAG-System lässt sich damit grundsätzlich in die beiden Schritte Retrieval und anschließende kontextgestützte Generierung unterteilen.

2.2.1 Sparse und Dense Retrieval

Eine zentrale Komponente eines RAG-Systems ist das Retrieval, das aus einer Dokumentensammlung die für eine Anfrage relevanten Inhalte bestimmen soll. Hierfür können unterschiedliche Repräsentationen von Anfrage und Dokument verwendet werden. Klassische Information-Retrieval-Verfahren arbeiten mit *sparse representations*, bei denen Texte über hochdimensionale, überwiegend nicht belegte Merkmalsvektoren repräsentiert werden. Ein etabliertes Verfahren ist BM25, das die Relevanz eines Dokuments unter anderem anhand der Häufigkeit von Suchbegriffen, ihrer Verteilung innerhalb der Dokumentensammlung und der Dokumentlänge bewertet [18]. Sparse Retrieval ist dadurch insbesondere auf lexikalische Übereinstimmungen zwischen Anfrage und Dokument ausgerichtet.

Dense Retrieval überführt Anfragen und Dokumente dagegen in niedrigdimensionale, dichte Vektorrepräsentationen. Karpukhin et al. zeigen mit *Dense Passage Retrieval* (DPR), dass Anfragen und Passagen durch getrennte Encoder in einen gemeinsamen Vektorraum abgebildet und anhand der Ähnlichkeit ihrer Repräsentationen verglichen

werden können [13]. Das Retrieval ist damit nicht ausschließlich von identischen Begriffen abhängig, sondern kann über gelernte Repräsentationen auch semantische Beziehungen zwischen unterschiedlich formulierten Anfragen und Passagen berücksichtigen.

Sparse und Dense Retrieval weisen damit unterschiedliche Eigenschaften auf. Luan et al. stellen heraus, dass sparse Repräsentationen insbesondere bei der präzisen Erkennung von Begriffen Vorteile besitzen, während Dense-Retrieval-Verfahren semantische Ähnlichkeiten zwischen verwandten Konzepten abbilden können [15]. Die Autoren untersuchen daher auch hybride Ansätze, in denen beide Repräsentationsformen miteinander kombiniert werden.

2.2.2 Hybrid Retrieval und Reciprocal Rank Fusion

Beim Hybrid Retrieval werden Ergebnisse verschiedener Retrieval-Verfahren zusammengeführt. In einem Sparse-Dense-Hybrid können für dieselbe Anfrage beispielsweise ein lexikalisches und ein semantisches Ranking erzeugt werden. Ziel ist es, die unterschiedlichen Eigenschaften beider Retrievalformen gemeinsam zu nutzen [15]. Da beide Verfahren zunächst eigenständige Ranglisten erzeugen, wird ein Verfahren zur Fusion der Rankings benötigt.

Eine Möglichkeit hierfür stellt die von Cormack et al. vorgestellte *Reciprocal Rank Fusion* (RRF) dar [1]. RRF kombiniert die Rangpositionen eines Dokuments über mehrere Ergebnislisten hinweg, ohne dass die von den einzelnen Retrieval-Verfahren erzeugten Scores auf einer gemeinsamen Skala liegen müssen. Der RRF-Score eines Dokuments d ergibt sich aus

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}, \quad (1)$$

wobei R die Menge der einbezogenen Rankings, $r(d)$ die Position des Dokuments d im jeweiligen Ranking und k eine Konstante zur Begrenzung des Einflusses hoher Rangpositionen bezeichnet [1]. Dokumente, die in einem oder mehreren Retrieval-Verfahren weit oben eingeordnet werden, erhalten dadurch einen entsprechend höheren kombinierten Score. Auf Basis dieses Scores entsteht eine gemeinsame Rangfolge der Kandidaten.

2.2.3 Cross-Encoder-Reranking

Das erste Retrieval reduziert eine große Dokumentensammlung auf eine begrenzte Menge potenziell relevanter Kandidaten. Diese Kandidaten können anschließend durch ein rechenintensiveres Modell erneut bewertet werden. Nogueira und Cho zeigen am Beispiel von BERT, dass ein Sprachmodell für das Query-basierte Reranking bereits abgerufener Passagen eingesetzt werden kann [17]. Dabei werden Anfrage und jeweilige Passage gemeinsam verarbeitet, sodass deren Beziehung unmittelbar in die Relevanzbewertung einfließen

kann. Ein solches Vorgehen wird als Cross-Encoder-Reranking bezeichnet.

Im Unterschied zu Retrieval-Verfahren, bei denen Dokumentrepräsentationen vorab berechnet und effizient über große Dokumentbestände durchsucht werden können, muss ein Cross-Encoder jedes Query-Dokument-Paar separat verarbeiten. Der höhere Berechnungsaufwand begrenzt seinen Einsatz auf eine zuvor bestimmte Kandidatenmenge. Reranking bildet deshalb typischerweise eine nachgelagerte Stufe der Retrieval-Pipeline.

Abbildung 1 fasst die beschriebenen Komponenten zusammen. Eine Anfrage wird zunächst parallel durch Sparse und Dense Retrieval verarbeitet. Die resultierenden Rankings können mittels RRF fusioniert und die verbleibenden Kandidaten anschließend durch einen Cross-Encoder neu bewertet werden. Die ausgewählten Inhalte bilden schließlich den zusätzlichen Kontext für die Generierung durch das LLM. Die konkrete technische Umsetzung und Konfiguration dieser Komponenten im untersuchten System wird in Kapitel 3 beschrieben.

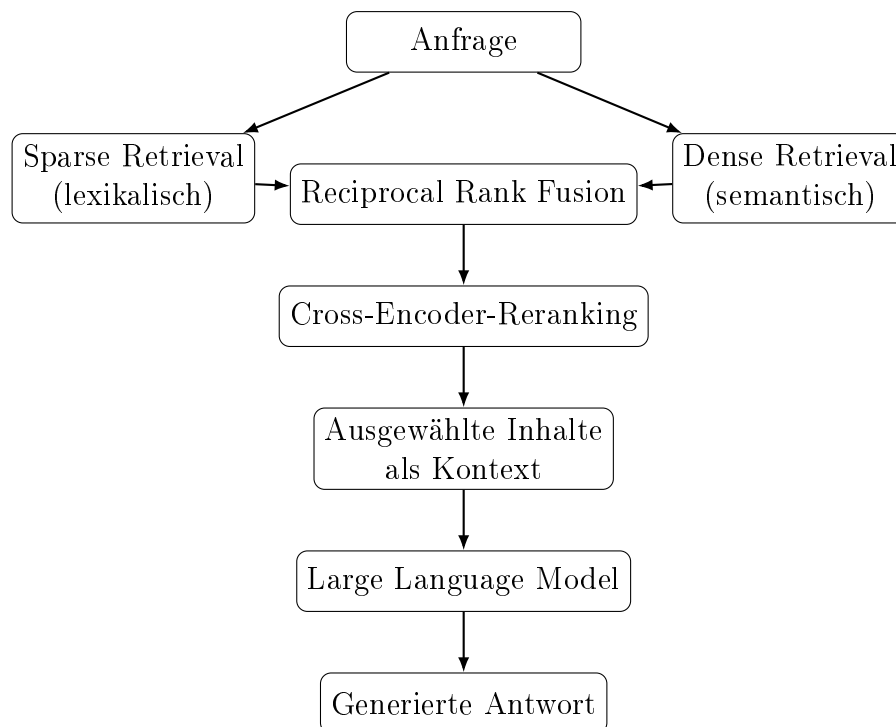


Figure 1: Schematischer Ablauf einer hybriden RAG-Pipeline mit Reranking.

2.3 Evaluation LLM-basierter Dialogsysteme

Die Bewertung von LLM-basierten Dialogsystemen unterscheidet sich von Aufgaben, für die eine eindeutig definierte Zielausgabe vorliegt. Insbesondere bei offenen Dialogen können mehrere inhaltlich und sprachlich unterschiedliche Antworten eine gestellte Anforderung erfüllen. Für solche Ausgaben kann ein Large Language Model selbst als Evaluator eingesetzt werden. Dieses Verfahren wird als *LLM-as-a-Judge* bezeichnet.

2.3.1 LLM-as-a-Judge

Beim LLM-as-a-Judge wird ein Large Language Model eingesetzt, um die Ausgabe eines zu untersuchenden LLM-Systems anhand vorgegebener Bewertungskriterien zu beurteilen. Zheng et al. untersuchen diesen Ansatz für die Evaluation von Chat-Assistenten und verwenden leistungsfähige LLMs sowohl zur Bewertung einzelner Antworten als auch zum Vergleich verschiedener Antworten [20]. Damit übernimmt das bewertende Modell eine Funktion, die andernfalls beispielsweise durch menschliche Evaluatoren ausgeführt werden müsste.

Der Ansatz ermöglicht insbesondere die Bewertung von Eigenschaften, die sich nicht ausschließlich durch einen Vergleich mit einer festgelegten Referenzantwort bestimmen lassen. Die Bewertung bleibt jedoch vom eingesetzten Judge-Modell und vom Bewertungsverfahren abhängig. Zheng et al. identifizieren unter anderem einen *Position Bias*, bei dem die Position einer Antwort das Urteil beeinflussen kann, einen *Verbosity Bias*, der ausführlichere Antworten begünstigen kann, sowie einen *Self-Enhancement Bias* zu Gunsten von Antworten verwandter Modelle [20]. LLM-as-a-Judge ist daher nicht als objektive Messung zu verstehen, sondern als modellbasierte Evaluationsmethode, deren Ausgestaltung bei der Interpretation der Ergebnisse berücksichtigt werden muss.

2.3.2 Multi-Turn-Evaluation und ConversationalTestCase

Bei einem Dialogsystem hängt die Qualität einer Antwort nicht ausschließlich von der unmittelbar vorhergehenden Eingabe ab. Aussagen, Anforderungen oder Informationen aus früheren Gesprächsschritten können für spätere Antworten weiterhin maßgeblich sein. Eine isolierte Bewertung einzelner Input-Output-Paare kann deshalb Eigenschaften wie die Berücksichtigung des Gesprächskontexts oder ein konsistentes Verhalten über den gesamten Dialog nur eingeschränkt erfassen. DeepEval unterscheidet hierzu zwischen Single-Turn- und Multi-Turn-Evaluation und verwendet für letztere den `ConversationalTestCase` [7].

Ein `ConversationalTestCase` repräsentiert eine vollständige Konversation als Folge von `Turn`-Objekten. Ein `Turn` enthält insbesondere die jeweilige Rolle, beispielsweise `user` oder `assistant`, sowie den Inhalt der Nachricht. Zusätzlich kann ein `Conversational Test Case` Angaben zum Szenario, zum erwarteten Ergebnis, zum verfügbaren Kontext oder zur vorgesehenen Rolle des Chatbots enthalten [7]. Die Konversation bildet damit die gemeinsame Evaluationsgrundlage für Metriken, deren Bewertung sich über mehrere Interaktionen erstreckt.

Diese Betrachtung ist insbesondere für Tutor-Chatbots erforderlich. Ein einzelner `Turn` kann für sich betrachtet einer vorgesehenen Tutorstrategie entsprechen, während das System diese Strategie in späteren Gesprächsschritten nicht fortführt. Die Multi-Turn-Evaluation ermöglicht deshalb die Untersuchung, ob gewünschte Eigenschaften über den

Gesprächsverlauf hinweg erhalten bleiben.

2.3.3 Conversational G-Eval

Für Anforderungen, für die keine vorgegebene Metrik existiert, stellt DeepEval mit `ConversationalGEval` eine anpassbare LLM-as-a-Judge-Metrik für Multi-Turn-Konversationen bereit [3]. Hierzu wird ein natürlichsprachliches Bewertungskriterium definiert, anhand dessen ein Judge-Modell die im `ConversationalTestCase` enthaltene Konversation beurteilt. Im Gegensatz zu einer auf einzelne Antworten beschränkten Bewertung kann damit das Verhalten über den gesamten Gesprächsverlauf Gegenstand des Bewertungskriteriums sein.

Dadurch können anwendungsspezifische Qualitätsanforderungen operationalisiert werden, für die keine allgemeingültige Metrik vorhanden ist. Im Kontext eines Tutors können solche Kriterien beispielsweise das didaktische Verhalten oder die Einhaltung bestimmter Gesprächsstrategien betreffen. Die konkrete Formulierung dieser Kriterien sowie ihre Verwendung für die Evaluation des in dieser Arbeit untersuchten Tutors werden im Methodikkapitel beschrieben.

2.3.4 Vordefinierte Multi-Turn-Metriken

Neben frei definierbaren Kriterien stellt DeepEval vordefinierte Metriken für wiederkehrende Qualitätsdimensionen von Multi-Turn-Systemen bereit. Diese arbeiten auf einem `ConversationalTestCase` und berücksichtigen den Gesprächskontext bei der Bewertung [6].

Die *Turn Relevancy* bewertet, ob die Antworten des Assistenten im jeweiligen Gesprächskontext relevant sind. Hierfür werden Assistant-Turns gemeinsam mit vorausgehenden Gesprächsschritten betrachtet und danach beurteilt, ob die jeweilige Antwort zum bisherigen Dialog passt [11]. Die *Role Adherence* untersucht dagegen, ob der Assistent über den Dialog hinweg der in `chatbot_role` festgelegten Rolle entspricht [8]. Mit *Knowledge Retention* wird bewertet, ob im Gespräch eingeführte Informationen in späteren Turns berücksichtigt und nicht widersprüchlich wiedergegeben werden. Die *Conversation Completeness* untersucht, ob die im Gespräch auftretenden Nutzerbedürfnisse über den gesamten Verlauf adressiert werden [2], [5].

Eine weitere vordefinierte Multi-Turn-Metrik ist die *Goal Accuracy*. Sie bewertet, in welchem Maß das System die aus den Nutzeräußerungen abgeleiteten Ziele erreicht. Hierzu werden die vom System gewählten Schritte sowie deren Eignung zur Erreichung des jeweiligen Ziels betrachtet [4]. Die Metrik berücksichtigt damit sowohl die Zielerreichung als auch die Qualität und Einhaltung des dafür verfolgten Vorgehens.

Für dialogbasierte RAG-Systeme stellt DeepEval darüber hinaus Multi-Turn-Metriken bereit, welche die Verwendung des abgerufenen Kontexts bewerten. *Turn Faithfulness* untersucht, ob die Aussagen des Assistenten durch den für den jeweiligen Turn bereitgestellten Retrieval-Kontext gestützt werden [10]. *Turn Contextual Relevancy* bewertet

dagegen, in welchem Umfang der abgerufene Kontext für die jeweilige Nutzereingabe relevant ist [9]. Während *Turn Faithfulness* damit die Verwendung des bereitgestellten Wissens bei der Antwortgenerierung betrachtet, richtet sich *Turn Contextual Relevancy* auf die Qualität der zuvor abgerufenen Inhalte.

Die vordefinierten Metriken und Conversational G-Eval erfüllen damit unterschiedliche Funktionen. Während die vordefinierten Metriken festgelegte Qualitätsdimensionen wie Relevanz, Rollentreue oder Retrieval-Qualität abbilden, ermöglicht Conversational G-Eval die Definition anwendungsspezifischer Bewertungskriterien. Welche dieser Metriken für die Untersuchung des Systemprompts und der RAG-Pipeline eingesetzt werden, wird im weiteren Verlauf der Arbeit methodisch begründet.

3 Untersuchungsgegenstand

Die Untersuchung betrachtet sowohl das Tutorverhalten als auch die Bereitstellung des fachlichen Kontexts. RQ1 adressiert den Vergleich unterschiedlicher Systemprompt-Varianten hinsichtlich ihres sokratischen Tutorverhaltens, während RQ2 die untersuchten RAG-Konfigurationen gegenüberstellt. Die Grundlage beider Untersuchungen ist die im Folgenden dargestellte RAG-Pipeline.

3.1 Die RAG-Pipeline

Als Untersuchungsgegenstand dient eine Open-WebUI-kompatible RAG-Pipeline, die OpenAI-kompatible Chat-Completion-Anfragen verarbeitet und Qdrant als Vektordatenbank für das Retrieval verwendet. Die Pipeline unterstützt Dense, Sparse und Hybrid Retrieval sowie Reciprocal Rank Fusion (RRF) und optionales Cross-Encoder-Reranking. Darüber hinaus können Kontextinformationen aus Moodle in die Verarbeitung einbezogen werden.

Der grundsätzliche Ablauf ist in Abbildung 2 dargestellt. Nach Eingang einer Nutzeranfrage werden zunächst die Request-Parameter und die aktuelle Frage bestimmt. Abhängig von der gewählten Retrieval-Konfiguration wird die Anfrage anschließend über Dense Retrieval, Sparse Retrieval oder eine Kombination beider Verfahren verarbeitet. Im hybriden Modus werden Dense- und Sparse-Repräsentationen parallel erzeugt. Die resultierenden Rankings werden mittels RRF zusammengeführt. Dabei wird, sofern verfügbar, die serverseitige Fusion durch Qdrant verwendet; andernfalls erfolgt die Fusion lokal innerhalb der Pipeline.

Die resultierenden Kandidaten werden zusammengeführt und gegebenenfalls dedupliziert. Optional kann anschließend ein Cross-Encoder die Kandidaten erneut bewerten und neu sortieren. Nach dem Retrieval beziehungsweise Reranking werden ein Score-Threshold sowie ein finales Trefferlimit angewendet. Die verbleibenden Chunks bilden die Grundlage für den RAG-Kontext. Dabei begrenzt ein Kontextbudget die Menge der tatsächlich an das Sprachmodell übergebenen Inhalte, sodass nicht zwingend jeder finale Retrieval-Treffer Bestandteil des LLM-Kontexts ist.

Für die Generierung wird der Prompt aus mehreren Bestandteilen zusammengesetzt. Im RAG-Pfad werden zunächst der aktive Systemprompt, anschließend ein optionaler Moodle-Kontext und der aus dem Retrieval erzeugte RAG-Kontext eingefügt. Darauf folgen die bereinigte bisherige Gesprächshistorie und die aktuelle Nutzerfrage. Die Pipeline ermöglicht damit, sowohl retrieved Informationen als auch den bisherigen Gesprächsverlauf bei der Generierung einer Tutorantwort zu berücksichtigen.

Version 1.7.0 stellt zusätzlich einen Verbose-Modus für Non-Streaming-Anfragen bereit. Dieser gibt unter anderem Informationen über die Vektorisierung, die Retrieval- und Fusionsergebnisse, das Reranking, die tatsächlich an das LLM übergebenen Nachrichten sowie die rohe und finale Modellantwort strukturiert aus. Der Verbose-Modus greift dabei

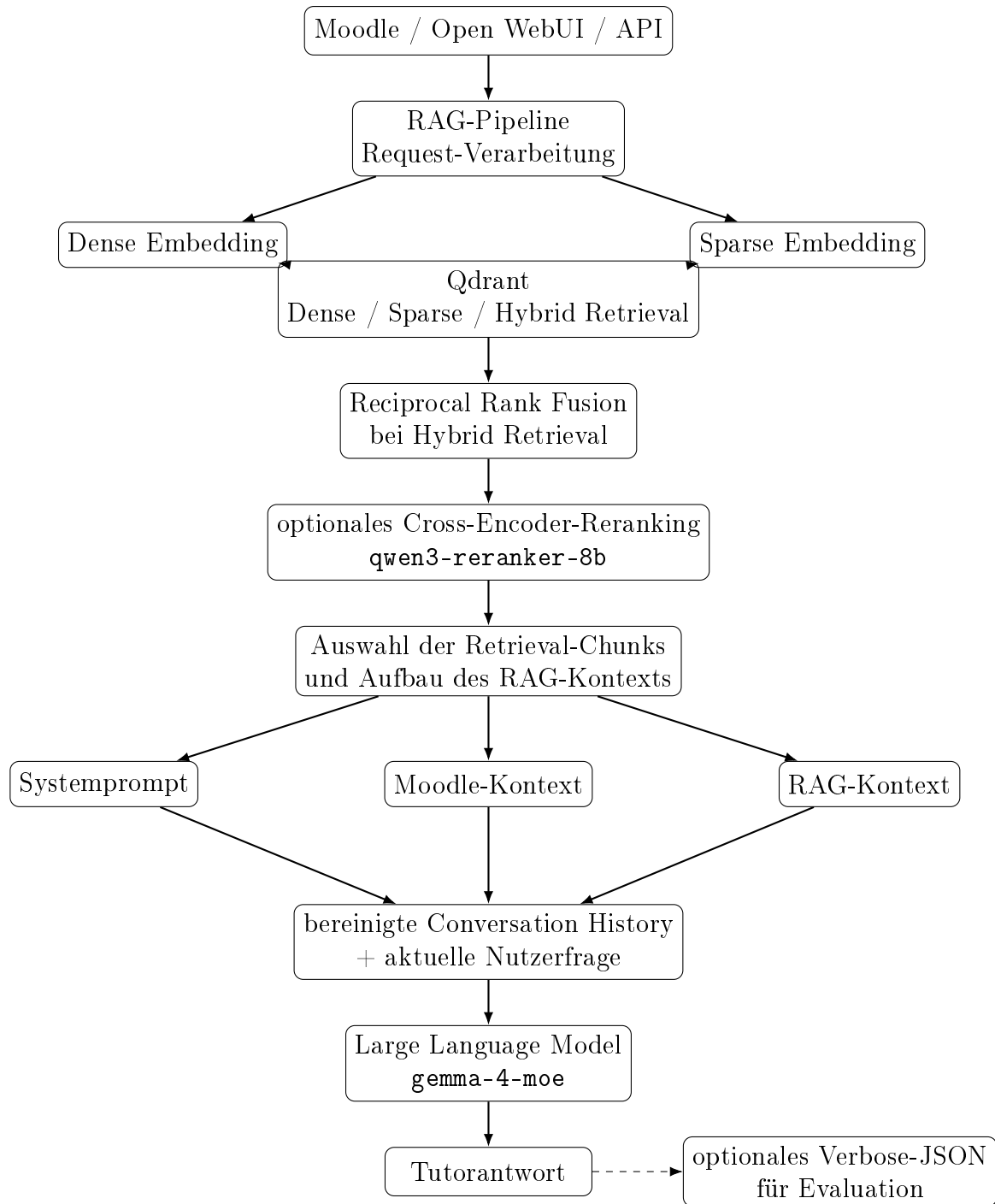


Figure 2: Architektur und Verarbeitungsschritte der untersuchten RAG-Pipeline in Version 1.7.0.

auf bereits während der regulären Verarbeitung erzeugte Daten zurück und führt keine zusätzlichen Retrieval-, Embedding-, Reranking- oder LLM-Aufrufe aus.

Dadurch können die einzelnen Verarbeitungsschritte für die spätere Evaluation nachvollzogen werden. Die für die Untersuchung variierten Retrieval- und Reranking-Konfigurationen werden in Abschnitt 3.3 beschrieben.

3.2 Die Prompt-Varianten für RQ1

Zur Beantwortung von RQ1 werden vier Varianten des Systemprompts miteinander verglichen. Sie unterscheiden sich hinsichtlich des Umfangs der Instruktionen, der Strukturierung des sokratischen Dialogs und der Anpassung an den konkreten Einsatz im Elektrotechniklabor. Dadurch soll untersucht werden, welchen Einfluss die Ausgestaltung des Systemprompts auf das resultierende Tutorverhalten hat. Die vollständigen Wortlaute der verwendeten Prompts sind in Anhang A.1 dokumentiert.

Die *Baseline ohne Systemprompt* enthält keine zusätzliche Instruktion zur Rolle oder zum Verhalten des Modells. Sie dient als Referenz, um zu untersuchen, welches Tutorverhalten das Sprachmodell ohne explizite Vorgaben zum sokratischen Vorgehen zeigt.

Der *Minimale sokratische Prompt* definiert das Modell als sokratischen Tutor für Elektrotechnik. Vorgegeben wird lediglich, Gegenfragen anstelle direkter Antworten zu stellen und Studierende durch Fragen zur eigenständigen Erkenntnis zu führen. Weitere Regeln zum Ablauf des Gesprächs oder zum Umgang mit bestimmten Situationen werden nicht festgelegt.

Der *Strukturierte sokratische Prompt* konkretisiert den Ablauf des sokratischen Dialogs. Ausgangspunkt soll eine konkrete Erfahrung der studierenden Person sein, aus der schrittweise Begriffe, Urteile sowie zugrunde liegende Überzeugungen und Prinzipien herausgearbeitet werden. Der Tutor soll sich dabei weitgehend aus der direkten fachlichen Problemlösung heraushalten, Gegenperspektiven einbringen und die studierende Person durch Fragen zum eigenständigen Weiterdenken anregen. Zusätzlich enthält der Prompt die Vorgabe, die Lösung einer Quizfrage nicht direkt oder indirekt preiszugeben.

Der *Domänenspezifische In-Lab-Tutorprompt* überträgt das sokratische Tutorverhalten auf Situationen während eines Elektrotechniklabors. Neben der Förderung eigenständigen Denkens enthält er Vorgaben zur Berücksichtigung des Vorwissens, zur Entwicklung und Prüfung eigener Fehlerhypothesen sowie zur Begrenzung der Informationsmenge und der Anzahl der Impulse pro Antwort. Darüber hinaus definiert der Prompt Eskalationsregeln für sicherheitsrelevante oder durch die Laborbetreuung zu klärende Situationen.

Tabelle 1 fasst die wesentlichen Unterschiede der vier Varianten zusammen.

Die vier Varianten bilden damit unterschiedliche Grade der Steuerung durch den Systemprompt ab. Während die Baseline vollständig auf eine sokratische Instruktion

Table 1: Für RQ1 untersuchte Prompt-Varianten.

ID	Variante	Umfang	Schwerpunkt
A	Baseline ohne Systemprompt	keiner	Keine explizite Vorgabe eines sokratischen Tutorverhaltens.
B	Minimaler sokratischer Prompt	gering	Definition der Tutorrolle sowie Vorgabe von Gegenfragen und Förderung eigenständiger Erkenntnis.
C	Strukturierter sokratischer Prompt	hoch	Vorgegebener sokratischer Gesprächsablauf und Vermeidung direkter Lösungsweitergabe.
D	Domänenspezifischer In-Lab-Tutorprompt	hoch	Anpassung des Tutorverhaltens an das Elektrotechniklabor mit Hypothesenbildung, begrenzten Impulsen und Eskalationsregeln.

verzichtet und der minimale Prompt lediglich das Grundprinzip vorgibt, operationalisieren die beiden umfangreicheren Varianten das gewünschte Tutorverhalten stärker. Der strukturierte sokratische Prompt fokussiert dabei den Ablauf des sokratischen Dialogs, während der domänenspezifische In-Lab-Tutorprompt zusätzlich Anforderungen des konkreten Laborkontexts berücksichtigt. Die methodische Durchführung des Vergleichs wird in Kapitel 4 beschrieben.

3.3 RAG-Konfigurationen für RQ2

Zur Beantwortung von RQ2 werden unterschiedliche Konfigurationen der in Abschnitt 3.1 beschriebenen RAG-Pipeline untersucht. Die Varianten unterscheiden sich hinsichtlich der verwendeten Wissensbasis, des Retrieval-Verfahrens und des Einsatzes eines Cross-Encoder-Rerankings. Dadurch kann untersucht werden, welchen Einfluss diese Komponenten auf die Qualität des bereitgestellten Retrieval-Kontexts und der darauf basierenden Antworten haben.

Als Wissensbasis werden eine Collection mit Vorlesungsinhalten, eine Collection mit Laborinhalten sowie die Kombination beider Collections betrachtet. Für das Retrieval kommen Dense, Sparse und Hybrid Retrieval zum Einsatz. Beim hybriden Verfahren werden Dense und Sparse Retrieval kombiniert und die resultierenden Ranglisten mittels Reciprocal Rank Fusion zusammengeführt. In ausgewählten Konfigurationen wird anschließend zusätzlich das in Abschnitt 3.1 beschriebene Cross-Encoder-Reranking angewendet.

Neben den RAG-Konfigurationen wird eine *LLM-only*-Variante untersucht. Bei dieser wird keine Collection ausgewählt, sodass Retrieval und Reranking vollständig übersprungen werden. Sie dient als Referenz für die Antwortgenerierung ohne zusätzlich bereitgestellten Retrieval-Kontext.

Tabelle 2 zeigt die für RQ2 untersuchten Konfigurationen.

Table 2: Für RQ2 untersuchte RAG-Konfigurationen.

Variante	Wissensbasis	Retrieval	Reranking
Beide – Hybrid	Vorlesung + Labor	Hybrid	Nein
Beide – Sparse	Vorlesung + Labor	Sparse	Nein
Beide – Hybrid + Reranking	Vorlesung + Labor	Hybrid	Ja
Vorlesung – Hybrid + Reranking	Vorlesung	Hybrid	Ja
Labor – Hybrid + Reranking	Labor	Hybrid	Ja
Beide – Dense	Vorlesung + Labor	Dense	Nein
Labor – Dense	Labor	Dense	Nein
Vorlesung – Dense	Vorlesung	Dense	Nein
LLM-only	keine	kein Retrieval	Nein

Die Auswahl der Varianten ermöglicht mehrere Vergleichsperspektiven. Durch die Dense-, Sparse- und Hybrid-Konfigurationen werden unterschiedliche Retrieval-Ansätze gegenübergestellt. Die Varianten mit beiden Collections und mit getrennten Vorlesungs- beziehungsweise Laborinhalten erlauben zusätzlich eine Untersuchung des Einflusses der zugrunde liegenden Wissensbasis. Der Vergleich der hybriden Varianten mit und ohne Reranking dient schließlich dazu, den zusätzlichen Einfluss des Cross-Encoder-Rerankings zu betrachten. Die konkrete Durchführung der Evaluation und die hierfür verwendeten Metriken werden in Kapitel 4 beschrieben.

4 Methodik

Gegenstand der vorliegenden Untersuchung ist die Evaluation eines sokratischen KI-Tutors, der auf einer Retrieval-Augmented-Generation-Pipeline (RAG) aufsetzt. Ziel ist es, das Zusammenspiel von Prompting und Retrieval-Konfiguration systematisch zu bewerten. Die Untersuchung folgt einem explorativen, experimentellen Evaluationsdesign. Das konkrete Vorgehen zur getrennten Beantwortung von RQ1 und RQ2 wird im folgenden Abschnitt beschrieben.

4.1 Untersuchungsdesign

Die Evaluation ist als automatisierte, mehrstufige Dialogbewertung angelegt. Für jedes Testszenario wird ein Multi-Turn-Gespräch zwischen einer simulierten studierenden Person und dem RAG-gestützten Tutor erzeugt und anschließend anhand eines definierten Metrik-Sets bewertet. Die Wahl eines simulationsbasierten Multi-Turn-Designs ergibt sich unmittelbar aus dem Untersuchungsgegenstand: Da sokratisches Tutoring ein dialogisches, über mehrere Gesprächsrunden verlaufendes Vorgehen ist, lässt es sich nicht anhand einzelner Frage-Antwort-Paare, sondern nur im Verlauf eines Gesprächs angemessen beurteilen (vgl. Abschnitt 2.1).

Zur klaren Trennung der Forschungsfragen wurden zwei separate Evaluationslinien umgesetzt. In der ersten Linie (RQ1) wird bei konstant gehaltener RAG-Konfiguration der System-Prompt variiert. Bewertet wird die sokratische Qualität des Gesprächsverlaufs. In der zweiten Linie (RQ2) wird bei konstant gehaltenem Prompt die RAG-Konfiguration variiert und die *Retrieval-Qualität* bewertet. Durch die Beschränkung auf jeweils einen variierten Faktor je Linie können die beobachteten Effekte den Prompts bzw. den Konfigurationen zugeordnet werden.

4.2 Untersuchungsgegenstand und Datengrundlage

Das zu evaluierende System (System under Test) ist die in Abschnitt 2.2 beschriebene RAG-Pipeline, die intern das Modell `gemma-4-26B` als generierendes Sprachmodell nutzt. Der fachliche Wissenskorpus liegt in zwei Qdrant-Collections vor, die Vorlesungs- bzw. Labormaterial des Fachgebiets Elektronik der Hochschule enthalten.

Die Datengrundlage der Evaluation bilden nicht vorab gesammelte Dialoge, sondern synthetisch erzeugte Gespräche. Grundlage jedes Gesprächs ist ein Testszenario, das sich aus der Kombination von Themengebiet, Kompetenzniveau der studierenden Person (Anfänger bzw. Fortgeschritten) und einem charakteristischen Studierenden-Verhalten (etwa kooperativ, die Lösung einfordernd oder inhaltlich selbstsicher-fehlerhaft) zusammensetzt 3. Insgesamt ergeben sich daraus mehrere Szenarien je Auswertungslinie. Jedes Szenario wird als *Golden* modelliert, das die Ausgangssituation, eine Beschreibung der zu

simulierenden Persona sowie eine feste Initialfrage enthält. Die Initialfrage wird bewusst wortgleich vorgegeben und nicht generiert, um über alle Prompts und Konfigurationen hinweg denselben Gesprächseinstieg und damit die Vergleichbarkeit der Läufe sicherzustellen. Die daran anschließenden Folgefragen werden hingegen dynamisch simuliert.

Table 3: Dimensionen der Testszenarien. Die Kombination aus Themengebiet, Kompetenzniveau und Studierenden-Verhalten ergibt die simulierten Gespräche.

Dimension	Ausprägungen
Themengebiet	Sperrspannung/Durchbruch der Z-Diode; Spannungsstabilisierung mit Z-Diode; Ohmsches Gesetz und Maschenregel
Kompetenzniveau	Anfänger, Fortgeschritten
Studierenden-Verhalten	kooperativ; gibt schnell auf; fordert die Lösung ein; äußert eine Vermutung; selbstsicher, aber inhaltlich falsch

4.3 Evaluationsarchitektur

Da der Untersuchungsgegenstand ein Dialog ist, sind neben dem zu bewertenden System zwei weitere Rollen erforderlich. Die Architektur umfasst somit drei klar getrennte Modelle. Der Tutor ist die RAG-Pipeline gemma-4-26B und beantwortet jeden Gesprächszug. Der Studenten-Simulator übernimmt die Rolle der studierenden Person: Ausgehend von der im Golden festgelegten, wortgleichen Initialfrage erzeugt er auf Basis von Szenario, Persona und der jeweils vorangegangenen Tutor-Antwort dynamisch die weiteren Folgefragen des Gesprächs. Ohne diese Rolle würde kein Dialog entstehen, sondern lediglich ein einzelnes Frage-Antwort-Paar. Als Simulator wird das Modell **gpt-4o-mini** eingesetzt und der Judge bewertet den fertigen Dialog. Hierfür wird mit gpt-4.1 bewusst ein vom System unter Test verschiedenes Modell gewählt, um einen Selbstbewertungs-Bias zu vermeiden, bei dem ein Modell anteilig seine eigenen Ausgaben bewertet 2.3. Simulator und Judge laufen auf getrennten Endpunkten.

4.4 Verwendete Methoden und Werkzeuge

Die technische Umsetzung erfolgt in Python unter Verwendung des Evaluations-Frameworks DeepEval. Die Multi-Turn-Gespräche werden mit dem ConversationSimulator des Frameworks erzeugt. Ausgehend von der festen Initialfrage generiert der Simulator über eine festgelegte Anzahl von Runden fortlaufend Studierenden-Nachrichten, die über eine Callback-Schnittstelle an die RAG-Pipeline gesendet werden. Die Antwort bildet jeweils den näch-

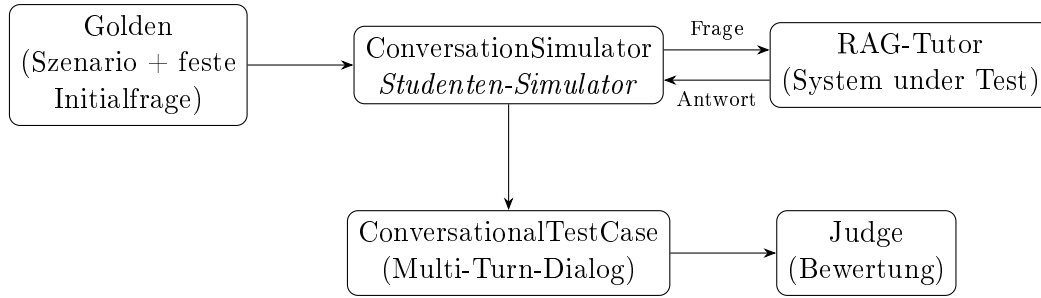


Figure 3: Ablauf der Dialog-Simulation: Aus einem Golden erzeugt der Conversation Simulator (in der Rolle der studierenden Person) fortlaufend Fragen an den RAG-Tutor. Der resultierende Multi-Turn-Dialog wird anschließend vom Judge bewertet.

sten Assistant-Turn. Die Zuordnung von Antworten zu Metriken erfolgt anschließend über ConversationalTestCase-Objekte des Frameworks.

Für die kontextbezogenen Metriken wird der tatsächlich abgerufene Retrieval-Kontext benötigt. Dieser wird über den Verbose-Modus der RAG-Pipeline gewonnen, der bei deaktiviertem Streaming eine strukturierte Antwort liefert, aus der die finalen Retrieval-Chunks samt Text extrahiert und den jeweiligen Antworten zugeordnet werden. Zwei Aufbereitungsschritte werden dabei transparent gemacht: Zum einen wird der abgerufene Kontext bereits beim Einlesen der RAG-Antwort auf eine handhabbare Länge begrenzt (maximal vier Chunks à 2000 Zeichen), um ein Überschreiten des Kontextfensters des Judge zu vermeiden. Diese Begrenzung betrifft die gespeicherten Gespräche direkt, da sie vor deren Speicherung angewendet wird. Zum anderen wird die mathematische Notation (LaTeX) unmittelbar vor der Bewertung für eine separate Kopie normalisiert, da sie andernfalls die strukturierte Ausgabe des Judge stört. Hiervon bleiben die gespeicherten Gespräche unverändert.

4.5 Bewertungskriterien

Die Bewertung ist entlang der beiden Forschungsfragen unterschiedlich ausgestaltet.

4.5.1 Sokratische Qualität (RQ1)

Sokratisches Tutoring ist ein didaktik-spezifisches Konstrukt, für das im verwendeten Framework keine vorgefertigten Metriken existieren. Es wird daher über eigene, mittels G-Eval operationalisierte Kriterien bewertet, die jeweils über explizite Bewertungsschritte und eine dreistufige Bewertungsskala (Rubric) verankert sind. Verwendet werden die Kriterien *Lösung nicht verraten*, *Sokratische Rückfragen*, *Schrittweise Lernprogression*, *Niveau-Anpassung*, *Fachliche Korrektheit*, *Verständnisförderung/Transfer* sowie *Respektvolle Kommunikation*. Ergänzend wird die im Framework enthaltene Metrik zur Rollentreue (*RoleAdherence*) als Gegenprüfung herangezogen, da sie die Treue zur im Testfall

hinterlegten sokratischen Rollendefinition misst und damit als einzige native Metrik das Konstrukt indirekt berührt⁴.

Bewusst nicht verwendet werden Metriken, die dem sokratischen Ziel widersprechen oder es verfehlen. Dies betrifft insbesondere eine Metrik zur direkten Zielerreichung, die die unmittelbare Beantwortung der Fachfrage bewertet und damit dem Prinzip, die Lösung gerade nicht vorwegzunehmen, entgegensteht, sowie eine generische Turn-Relevanz-Metrik, die legitimes sokratisches Umlenken als weniger relevant einstufen kann.

Zur Beantwortung von RQ1 werden die vier in Abschnitt 3.2 eingeführten Prompt-Varianten A–D verglichen, die in den Rohdaten den Dateien `no_Prompt` (A), `minimaler_sokrat` (B), `system_prompt` (C) und `stock_prompt` (D) entsprechen. Der vollständige Wortlaut der Prompts ist im Anhang dokumentiert.

Table 4: Bewertungskriterien der sokratischen Qualität (RQ1) mit ihrer literaturgestützten Fundierung. Sechs Kriterien werden als eigene ConversationalGEval-Metriken operationalisiert, die Rollentreue (RoleAdherence) wird als native Metrik ergänzend als Gegenprüfung herangezogen.

Kriterium	Gemessenes Konstrukt
Lösung nicht verraten	Führt zur eigenen Erkenntnis, statt die Lösung zu nennen; lässt eigene Denkarbeit.
Sokratische Rückfragen	Zielgerichtete, offene Fragen – keine Fragen „um der Frage willen“.
Schrittweise Lernprogression	Neues Wissen in kleinen Schritten auf Vorwissen aufbauend.
Niveau-Anpassung	Diagnostiziert das Niveau und unterrichtet daran angepasst.
Fachliche Korrektheit	Real korrekte Fachinhalte; keine erfundenen Formeln/Werte.
Verständnisförderung / Transfer	Betont wichtige Lernpunkte; fördert Verständnis statt Reproduktion.
Respektvolle Kommunikation	Ziel ist Lehren, nicht Bloßstellen; keine Beschämung.
RoleAdherence (nativ)	Gegenprüfung der Treue zur sokratischen Rollendefinition.

4.5.2 Retrieval-Qualität (RQ2)

Für den Vergleich der RAG-Konfigurationen werden die nativen Metriken *TurnContextualRelevancy* (Passung des abgerufenen Kontexts zur Frage) und *TurnFaithfulness* (Deckung der Antwort durch den abgerufenen Kontext) verwendet. Beide beziehen den zuvor extrahierten Retrieval-Kontext ein und eignen sich damit zur Unterscheidung von

Retrieval-Konfigurationen.

Table 5: Bewertungskriterien der Retrieval-Qualität (RQ2). Die beiden Metriken sind native DeepEval-Metriken und beziehen den tatsächlich abgerufenen `retrieval_context` ein. Die Abdeckung und die effektive Retrieval-Güte werden daraus abgeleitet.

Kennzahl		Herkunft	Gemessenes Konstrukt
TurnContextualRelevancy		nativ	Passung des abgerufenen Kontexts zur jeweiligen Frage.
TurnFaithfulness		nativ	Deckung der Antwort durch den abgerufenen Kontext (Grounding).
Retrieval-Abdeckung		abgeleitet	Anteil der Antworten mit tatsächlich abgerufenem Kontext (aus den Konversationsdaten).
Effektive Güte	Retrieval-	abgeleitet	Produkt aus Abdeckung und Kontext-Relevanz. Berücksichtigt, wie oft <i>und</i> wie gut Kontext abgerufen wird.

4.6 Durchführung

Die Durchführung erfolgt zweiphasig und je Prompt-Version bzw. Konfiguration getrennt: Zunächst werden alle Gespräche simuliert, anschließend werden sie bewertet. Diese Trennung erlaubt es, die netz- und laufzeitintensive Simulation gegen den RAG-Endpunkt einmalig durchzuführen und die Judge-Bewertung unabhängig davon zu wiederholen. Zur Sicherstellung der Reproduzierbarkeit werden die simulierten Gespräche je Konversation atomar zwischengespeichert. Über eine deterministische Konversations-ID wird ein unterbrochener Lauf beim Neustart fortgesetzt, ohne bereits erzeugte Gespräche neu zu berechnen. Die Bewertungsergebnisse werden fortlaufend je Metrik protokolliert und ebenfalls fortsetzbar geschrieben.

Für RQ1 werden vier Prompt-Varianten bei einer festen RAG-Konfiguration ausgewertet. Für RQ2 wird ein fester sokratischer Prompt mit einer Matrix aus neun RAG-Konfigurationen kombiniert, die sich in den einbezogenen Collections, im Retrieval-Modus (dense, sparse, hybrid) und im Einsatz des Cross-Encoder-Rerankings unterscheiden und zusätzlich eine reine LLM-Konfiguration ohne Retrieval als Referenz umfassen. Zur Steuerung der Endpunkt-Auslastung wird ein gemeinsamer Rate-Limiter eingesetzt. Transiente Fehler (etwa Rate-Limits oder Timeouts) werden mit ansteigender Wartezeit wiederholt, deterministische Fehler hingegen unmittelbar protokolliert.

Die Auswertung erfolgt anschließend automatisiert: Aus den protokollierten Rohdaten werden je Auswertungslinie die beschriebenen Kennzahlen berechnet und zur weiteren Analyse mit pandas (Aggregation, Pivot-Tabellen) und matplotlib (Balkendiagramme, Boxplots, Heatmaps) in Jupyter-Notebooks aufbereitet.

4.7 Gütekriterien und methodische Einschränkungen

Sämtliche Bewertungen sind Urteile eines LLM-Judge und damit modellabhängig. Ergebnisse verschiedener Judge-Modelle sind nicht unmittelbar vergleichbar. Die selbst definierten G-Eval-Kriterien sind projektspezifisch und wurden durch eine manuelle Nachbewertung einer Stichprobe abgesichert. Da ein Teil der Antworten ohne abgerufenen Kontext erzeugt wird, sind die kontextbezogenen Metriken nur eingeschränkt aussagekräftig. Aufgrund der begrenzten Anzahl an Szenarien je Konfiguration sind die Ergebnisse als explorativ zu verstehen.

4.8 Projekt-Setup

Die vollständige technische Umsetzung – Installation, Konfiguration der Endpunkte und Modelle, Ausführung von Simulation und Bewertung sowie Erweiterung um neue Prompt-Varianten, RAG-Konfigurationen oder Bewertungskriterien – ist im Projektrepositorium in einer begleitenden README dokumentiert. Diese ist bewusst auf Nachvollziehbarkeit und Wiederverwendbarkeit ausgelegt, sodass die Evaluation auch nach Abschluss dieser Arbeit reproduziert oder für andere Anwendungsfälle angepasst werden kann, ohne den Quellcode im Detail nachvollziehen zu müssen.

5 Ergebnisse

Dieses Kapitel berichtet die Ergebnisse der Evaluation entlang der beiden getrennten Auswertungslinien. Abschnitt 5.1 behandelt den Vergleich der Prompt-Varianten (RQ1) und Abschnitt 5.2 den Vergleich der RAG-Konfigurationen (RQ2). Abschnitt 5.3 ergänzt eine metrikbezogene, verhaltensbezogene, themenbezogene und niveaubezogene Querschnittsbetrachtung sowie eine qualitative Analyse der auffälligen Einzelfälle. Die Darstellung ist primär deskriptiv, wird jedoch um kritische Erklärungsansätze zu den identifizierten Verhaltensmustern des Modells ergänzt; die tiefere inhaltliche Einordnung sowie Diskussion der Befunde – einschließlich der Frage, welche der beobachteten Muster als didaktische Stärke und welche als methodisches Artefakt zu werten sind – erfolgt geschlossen in Kapitel 6.

Die Auswertungslinie für RQ1 beruht auf dem bereinigten Datensatz von insgesamt 114 Szenarien (Kombination aus Themengebiet, Kompetenzniveau und Studierendenverhalten). Diese verteilen sich auf jeweils 30 Durchläufe für die Varianten *Minimaler sokratischer Prompt*, *Baseline ohne Systemprompt* und *Domänenspezifischer In-Lab-Tutorprompt* sowie 24 Durchläufe (18 nach Bereinigung spezifischer Metriken) für den *Strukturierten sokratischen Prompt*. Alle Metriken sind auf den Wertebereich 0 bis 1 normiert. Als Bestehensgrenze der Einzelmetriken gilt der Schwellenwert 0,7. Die Pass-Rate bezeichnet den Anteil der Gespräche mit einem Score $\geq 0,7$.

Neben Mittelwerten werden, wo dies für die Beurteilung der Ergebnisstabilität relevant ist, auch Streuungsmaße und die Verteilung über Scorebänder berichtet.

5.1 RQ1 – Vergleich der Prompt-Varianten

Verglichen werden die vier eingeführten Prompt-Varianten *C* (Strukturierter sokratischer Prompt), *D* (Domänenspezifischer In-Lab-Tutorprompt), *B* (Minimaler sokratischer Prompt) und *A* (Baseline ohne Systemprompt) bei konstant gehaltener RAG-Konfiguration. Die Bewertung erfolgt über die G-Eval-Kriterien der sokratischen Qualität sowie die native Metrik Role Adherence als Gegenprüfung.

5.1.1 Ergebnisse je Kriterium

Tabelle 6 stellt den Mittelwert ($\emptyset$) und die Pass-Rate (PR, in %) je Kriterium und Prompt-Variante gegenüber. Abbildung 4 visualisiert die Mittelwerte als Balkendiagramm und Abbildung 5 verdichtet sie als Heatmap.

Bereits auf Ebene der Einzelkriterien zeigt sich ein klares, paradoxes Muster. Bei den allgemeinen, „weichen“ und sachlichen Qualitätskriterien – Fachliche Korrektheit, Schrittweise Lernprogression, Niveau-Anpassung und Respektvolle Kommunikation – liegen alle vier Varianten dicht beieinander auf einem sehr hohen Niveau. Hier erreicht die prompt-

Table 6: Mittelwert ($\emptyset$) und Pass-Rate (PR, in %) je Kriterium und Prompt-Variante.

Kriterium	C	D	B	A
Lösung nicht verraten	0,94 (94 %)	0,95 (97 %)	0,83 (77 %)	0,34 (13 %)
Sokratische Rückfragen	0,97 (100 %)	0,95 (97 %)	0,92 (93 %)	0,39 (30 %)
Schrittweise Lernprogression	0,91 (94 %)	0,91 (90 %)	0,90 (83 %)	0,98 (100 %)
Niveau-Anpassung	0,84 (78 %)	0,89 (80 %)	0,89 (80 %)	0,97 (100 %)
Fachliche Korrektheit	0,97 (100 %)	0,98 (100 %)	0,99 (100 %)	1,00 (100 %)
Verständnisförderung / Transfer	0,98 (100 %)	0,96 (100 %)	0,96 (100 %)	0,96 (97 %)
Respektvolle Kommunikation	0,94 (94 %)	0,95 (93 %)	0,94 (97 %)	1,00 (100 %)
Role Adherence (nativ)	0,94 (89 %)	0,92 (90 %)	0,93 (100 %)	0,16 (7 %)

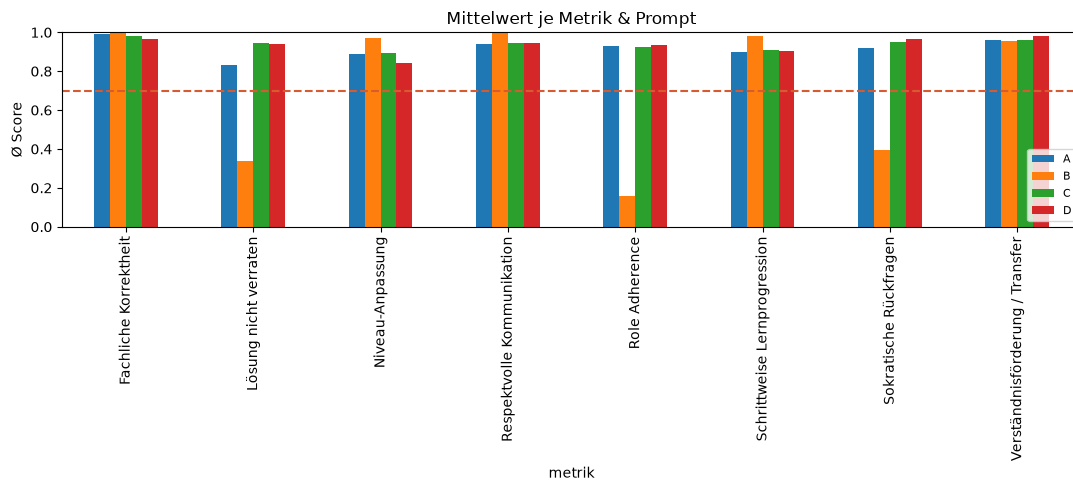


Figure 4: Mittelwert je Kriterium und Prompt-Variante. Die gestrichelte Linie markiert den Schwellenwert 0,7. (Legende: A = Baseline ohne Systemprompt, B = Minimaler sokratischer Prompt, C = Strukturierter sokratischer Prompt, D = Domänenspezifischer In-Lab-Tutorprompt)

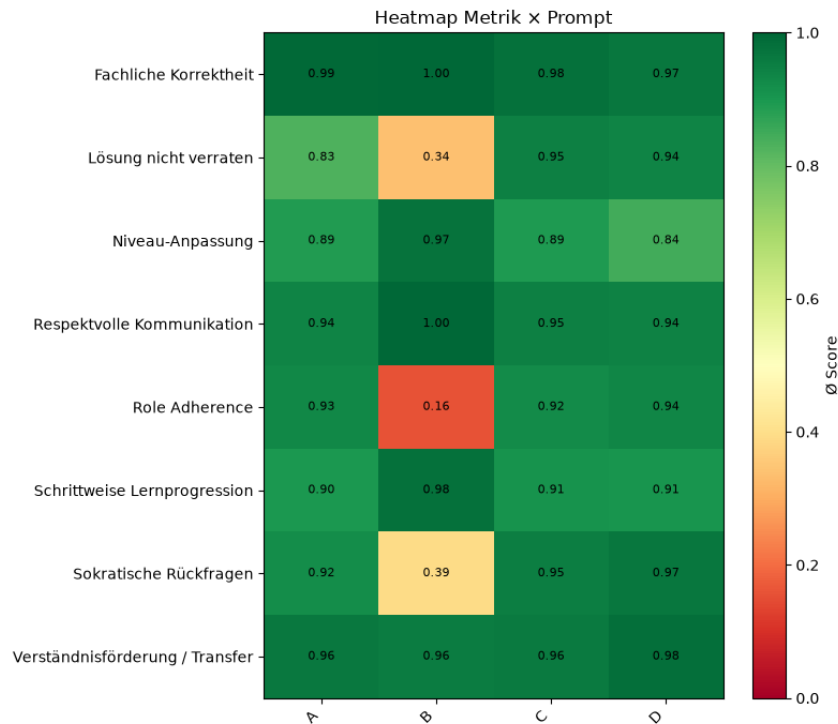


Figure 5: Mittelwerte je Kriterium und Prompt-Variante als Heatmap.

lose Referenz (*Baseline ohne Systemprompt*) sogar absolute Spitzenwerte und glänzt mit einer Pass-Rate von 100 % in fast allen diesen Kategorien.

Die Varianten trennen sich jedoch drastisch bei den sokratisch spezifischen Kriterien „Lösung nicht verraten“, „Sokratische Rückfragen“ und Role Adherence. Genau in diesen drei Kriterien stürzt *Baseline ohne Systemprompt* auf Werte von 0,34, 0,39 und 0,16 ab. Dies belegt eindrucksvoll den sogenannten „Helpful Assistant“-Bias, der durch das RLHF-Training (Reinforcement Learning from Human Feedback) der Basismodelle konditioniert ist: Ohne spezifisches Prompting agiert das System als hocheffizienter Lösungsautomat, der dem Nutzer schnellstmöglich die perfekte fachliche Antwort präsentiert, anstatt in der didaktischen Rolle eines Tutors zu verbleiben. Ein steuernder Prompt ist somit zwingende Bedingung für den pädagogischen Einsatz.

Gleichzeitig wird ein Zielkonflikt bei der Niveau-Anpassung deutlich: Je strikter das Modell durch einen Prompt (wie den *Strukturierter sokratischer Prompt*) gezwungen wird, die Lösung nicht zu verraten und stattdessen Rückfragen zu stellen, desto niedriger wird die Niveau-Anpassung bewertet (0,84). Einem Anfänger mit einer Gegenfrage zu begegnen, anstatt ihm die Lösung vorzurechnen, führt in der Evaluation oftmals zu Abzügen bei der Anpassungsfähigkeit.

5.1.2 Ergebnisstabilität und Streuung

Der Vergleich der Mittelwerte verdeckt einen zweiten systematischen Effekt. Mit zunehmender Instruktionstiefe steigen die didaktischen Werte nicht nur an, sie werden auch deutlich

stabiler. Tabelle 7 zeigt dies für die Standardabweichung je Kriterium.

Table 7: Standardabweichung je Kriterium und Prompt-Variante. Niedrigere Werte bedeuten konsistentere Ergebnisse.

Kriterium	C	D	B	A
Lösung nicht verraten	0,17	0,14	0,29	0,25
Sokratische Rückfragen	0,06	0,09	0,14	0,40
Schrittweise Lernprogression	0,17	0,16	0,15	0,05
Niveau-Anpassung	0,26	0,21	0,20	0,05
Fachliche Korrektheit	0,07	0,06	0,03	0,02
Verständnisförderung / Transfer	0,04	0,06	0,06	0,09
Respektvolle Kommunikation	0,12	0,13	0,13	0,00
Role Adherence (nativ)	0,16	0,16	0,11	0,23

Besonders ausgeprägt ist dieser Effekt bei den kritischen Kriterien. Der *Minimaler sokratischer Prompt* weist bei „Lösung nicht verraten“ (0,29) und „Sokratische Rückfragen“ (0,14) eine hohe Streuung auf. Dies deutet auf das Phänomen des „Attention Decay“ hin: Ein sehr kurzer Prompt verliert im Verlauf einer längeren Konversation, besonders wenn die studierende Person Frustration äußert oder Druck aufbaut, seine bindende Wirkung. Das Modell oszilliert dann stark zwischen sokratischem Verhalten und dem Verraten der Lösung. Die ausführlichen Prompts (*Strukturierter sokratischer Prompt* und *Domänenspezifischer In-Lab-Tutorprompt*) sorgen hingegen für ein signifikant konsistenteres Antwortverhalten in den didaktischen Metriken.

5.1.3 Gewichteter Gesamtscore und Gate-Compliance

Eine detaillierte Betrachtung der Gate-Verletzungen (die Vorgabe, dass „Lösung nicht verraten“ und „Fachliche Korrektheit“ gemeinsam einen Schwellenwert von $> 0,5$ erreichen müssen) ist äußerst aufschlussreich: Sämtliche Verletzungen – über alle Varianten hinweg – gehen fast ausschließlich auf das Kriterium „Lösung nicht verraten“ zurück. Die Fachliche Korrektheit fällt so gut wie nie ab und liegt im Gesamtmittelwert über alle Testläufe bei herausragenden 0,986. Die Einhaltung der didaktischen Kernanforderungen ist damit vollständig eine Frage der Lösungszurückhaltung, nicht der fachlichen Verlässlichkeit der RAG-Architektur.

5.1.4 Charakterisierung der einzelnen Varianten

Strukturierter sokratischer Prompt. Die ausführlichste Variante führt die Studierenden am konsequentesten durch den Dialog, was sich im Bestwert bei Sokratische Rückfragen (0,97 bei der geringsten Streuung von 0,06) widerspiegelt. Sie erzielt zudem Top-Werte bei der Verständnisförderung (0,98). Ihre einzige systematische Schwäche ist die Niveau-Anpassung (0,84), die als unvermeidbarer Trade-off für die harte didaktische Rollenbindung in Kauf genommen werden muss.

Domänenspezifischer In-Lab-Tutorprompt. Die Standardvariante erweist sich als exzellenter, ausbalancierter Allrounder. Sie brilliert beim Zurückhalten der Lösung (0,95 bei sehr geringer Streuung von 0,14) und verzeichnet ein besseres Gleichgewicht bei der Niveau-Anpassung (0,89) als der System-Prompt, während die sokratischen Rückfragen ebenfalls auf einem exzellenten Niveau (0,95) verbleiben.

Minimaler sokratischer Prompt. Der einzeilige Prompt erreicht bei Fachlicher Korrektheit (0,99) und Verständnisförderung (0,96) Top-Werte und zeigt, dass bereits eine minimale Instruktion den Tonfall des Modells anpasst. Die Lösungszurückhaltung bleibt mit 0,83 (und einer hohen Streuung von 0,29) jedoch die schwächste unter den instruierten Varianten, da das Modell unter starkem Nutzerdruck zu schnell einknickt.

Baseline ohne Systemprompt. Ohne steuernden Prompt bricht die Referenz bei den sokratisch-spezifischen Kriterien komplett ein (Role Adherence 0,16; Sokratische Rückfragen 0,39). Gleichzeitig erreicht sie bei Fachlicher Korrektheit (1,00) und Respektvoller Kommunikation (1,00) das absolute Maximum. Ohne Instruktion verhält sich das System wie ein freundlicher, fachlich perfekter Erklärer, der die Aufgabe unmittelbar löst – und sabotiert damit den eigenen Lernprozess.

5.2 RQ2 – Vergleich der RAG-Konfigurationen

Zur Beantwortung der zweiten Forschungsfrage (RQ2) wird die RAG-Konfiguration über neun verschiedene Varianten hinweg variiert. Um Wechselwirkungen mit der didaktischen Steuerung auszuschließen, wurde der Prompt in diesen Testreihen konstant gehalten (es wurde durchgehend der leistungsstärkste Prompt, der *Strukturierter sokratischer Prompt*, verwendet).

Die Evaluierung der Retrieval-Qualität erfolgt über zwei zentrale Achsen: Zum einen die **Retrieval-Abdeckung** (der Anteil der System-Antworten, für die tatsächlich Kontext aus der Datenbank abgerufen wurde), und zum anderen die kontextbezogenen DeepEval-Qualitätsmetriken **Turn Contextual Relevancy** (Wie relevant ist der gefundene Kontext für die Beantwortung der Frage?) und **Turn Faithfulness** (Wie stark hält sich das Sprachmodell an den bereitgestellten Kontext, ohne zu halluzinieren?).

5.2.1 Datenlage und Retrieval-Abdeckung

Die Retrieval-Abdeckung ist der grundlegendste label-freie Indikator für die Leistungsfähigkeit einer RAG-Pipeline. Tabelle 8 zeigt den zentralen RQ2-Befund. Die Abdeckung unterscheidet sich zwischen den Konfigurationen drastisch und wird nahezu vollständig durch den Retrieval-Modus und das Reranking bestimmt: Sparse-Retrieval erreicht die volle Abdeckung (100 %), die drei Hybrid-Konfigurationen *mit* Cross-Encoder-Reranking liegen hoch (75–94 %), während Dense-Retrieval auf 65–74 % abfällt.

Bei der Interpretation ist zu beachten, dass die Abdeckung den Anteil der Antworten

Table 8: Retrieval-Abdeckung je RAG-Konfiguration (Anteil der Antworten mit tatsächlich abgerufenem Kontext), absteigend sortiert.

Konfiguration	Modus	Rerank	Antworten	mit Kontext	Abdeckung
beide_hybrid	hybrid	–	207	207	100 %
beide_sparse	sparse	–	207	207	100 %
beide_hybrid_rerank	hybrid	ja	208	195	94 %
vorlesung_hybrid_rerank	hybrid	ja	208	191	92 %
labor_hybrid_rerank	hybrid	ja	201	159	79 %
beide_dense	dense	–	209	154	74 %
labor_dense	dense	–	209	145	69 %
vorlesung_dense	dense	–	204	133	65 %
llm_only	–	–	208	0	0 %

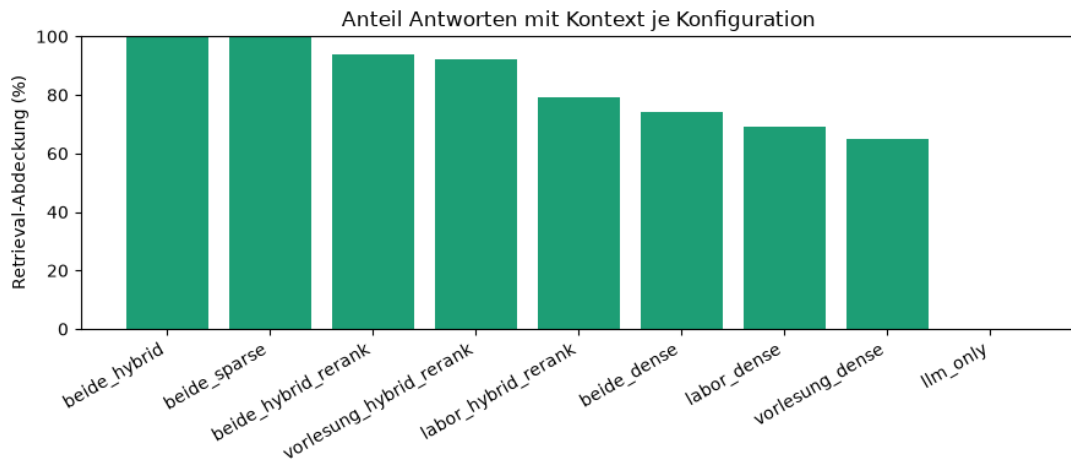


Figure 6: Anteil Antworten mit Kontext je RAG-Konfiguration.

erfasst, deren abgerufener Kontext einen festen Score-Schwellenwert überschreitet. Reine Sparse-Suche (z. B. BM25) basiert auf exaktem Keyword-Matching. In einem fachspezifischen Kontext wie dem Elektroniklabor, in dem Studierende spezifische Begriffe verwenden, schlägt diese Methode zuverlässig an. Dense-Retrieval (semantische Vektorsuche) agiert hingegen deutlich konservativer und erreicht selbst im besten Fall (*beide_dense*) nur 74 % Abdeckung.

Die Zuschaltung eines Cross-Encoder-Rerankers führt bei den Hybrid-Ansätzen zu einem leichten Abfall der Abdeckung (z. B. von 100 % auf 94 % bei *beide_hybrid_rerank*). Dies ist jedoch als Qualitätsgewinn zu werten: Der Reranker fungiert als scharfer Präzisionsfilter. Er verwirft jene Chunks, die beim initialen Retrieval zwar Keywords enthielten, im inhaltlichen Kontext der Frage aber irrelevant („False Positives“) waren.

5.2.2 Kontextbezogene Metriken und effektive Scores

Die tiefergehende Evaluierung der abgerufenen Inhalte wurde exemplarisch für fünf Konfigurationen berechnet. Tabelle 9 stellt die Mittelwerte der RAG-spezifischen DeepEval-Metriken gegenüber.

Table 9: Mittelwerte der RAG-Metriken für ausgewählte Konfigurationen.

Konfiguration	Turn Contextual Relevancy	Turn Faithfulness
beide_dense	0,504	0,987
beide_sparse	0,276	1,000
labor_dense	0,514	0,977
vorlesung_dense	0,416	1,000
llm_only	1,000*	0,990*

* Artefakt bei fehlendem Kontext.

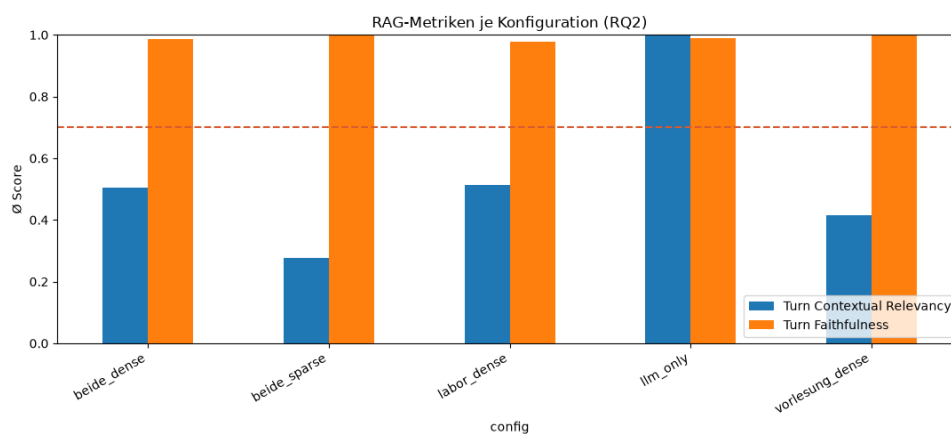


Figure 7: RAG-Metriken (Contextual Relevancy und Faithfulness) je Konfiguration.

Zwei Aspekte sind hier hervorzuheben: Erstens beweisen die exzellenten Turn Faithfulness-Scores (durchgängig $> 0,97$), dass das RAG-System seine Kernaufgabe als „Wahrheit-sanker“ erfüllt. Wenn Kontext bereitgestellt wird, hält sich das Sprachmodell strikt an

diese Fakten und die Halluzinationsrate tendiert gegen null. Zweitens liegt die Turn Contextual Relevancy (z.B. 0,514 bei *labor_dense*) nur auf einem mäßigen Niveau, bei Sparse-Suche sogar noch niedriger (0,276). Dies bedeutet, dass der abgerufene Text zwar die gesuchte Antwort enthält, gleichzeitig aber noch sehr viel irrelevantes „Rauschen“ in den Prompt lädt.

Die Tabelle 9 zeigt, dass die Baseline *llm_only* (die gänzlich ohne RAG-Kontext arbeitet) eine Contextual Relevancy von 1,000 erreicht. Die Fehleranalyse entlarvt diese scheinbaren Bestwerte als tiefgreifendes Bewertungs-Artefakt des DeepEval-Frameworks. Wenn kein Kontext existiert, kann das Modell ihm gegenüber auch nicht untreu oder irrelevant sein. Die Metrik kollabiert und vergibt fälschlicherweise perfekte Scores.

Eine Überprüfung der Konversationsdaten zeigte, dass abgesehen von der erwartungsgemäß vollständig kontextfreien Baseline *llm_only* keine Konversation gänzlich ohne Retrieval-Kontext blieb. Ein Ausschluss vollständig kontextfreier Dialoge war somit nicht erforderlich. Da die vorliegende Implementierung nur den aggregierten Score pro Konversation persistiert, lässt sich der Anteil des Artefakts bei teilweise gegroundeten Dialogen jedoch nicht nachträglich isolieren.

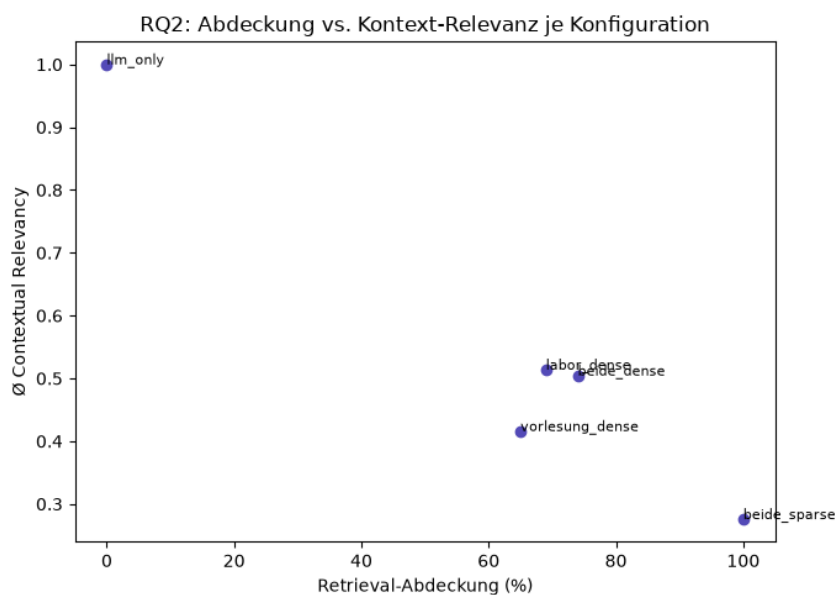


Figure 8: Effektive Retrieval-Güte: Abdeckung vs. Kontext-Relevanz je Konfiguration.

5.3 Querschnitt: Verhalten, Thema und Niveau

5.3.1 Verteilung der Einzelscores

Tabelle 10 zeigt für *Strukturierter sokratischer Prompt* die Verteilung der Einzelscores je Kriterium über drei Score-Bänder. Sie belegt, dass die Ergebnisse überwiegend stabil und nicht durch Ausreißer erzeugt sind: In sechs der acht Kriterien liegen alle gefilterten

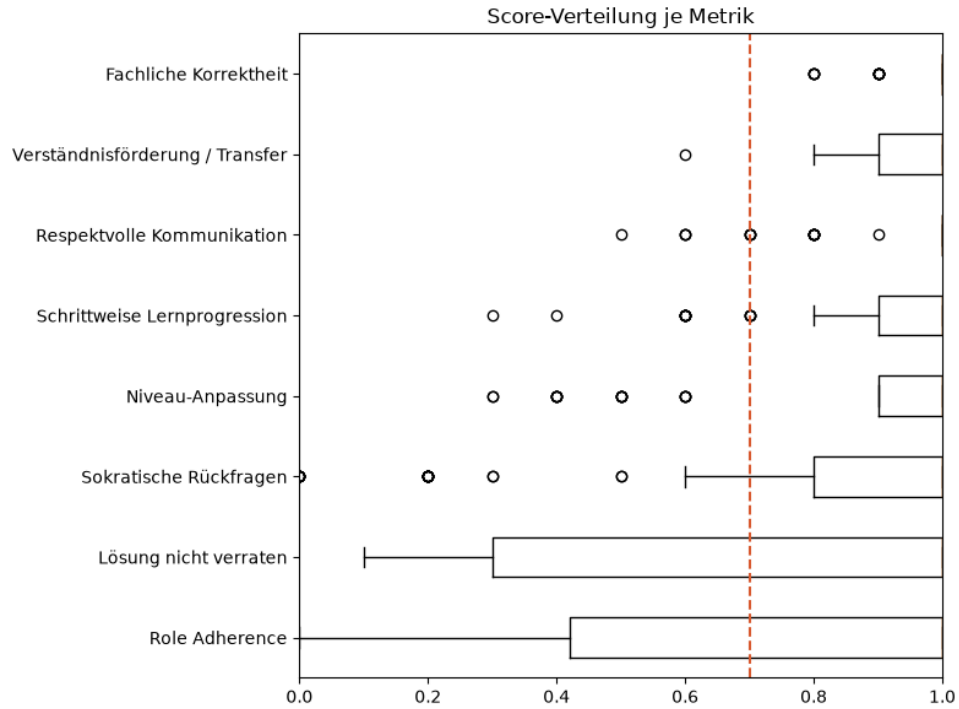


Figure 9: Verteilung der Einzelscores je Kriterium, über alle vier Varianten gepoolt.

Gespräche im obersten Band ($\geq 0,7$). Die Niveau-Anpassung ist die einzige Metrik mit einer nennenswerten Zahl schwacher Fälle.

Table 10: Verteilung der Einzelscores je Kriterium für Strukturierter sokratischer Prompt: Anzahl Gespräche mit Score $< 0,5$ / $0,5-0,7$ / $\geq 0,7$.

Kriterium	$< 0,5$	$0,5-0,7$	$\geq 0,7$
Sokratische Rückfragen	0	0	18
Lösung nicht verraten	0	1	17
Schrittweise Lernprogression	0	1	17
Niveau-Anpassung	4	1	13
Fachliche Korrektheit	0	0	18
Respektvolle Kommunikation	0	0	18
Verständnisförderung / Transfer	0	0	18
Role Adherence	0	0	18

5.3.2 Ergebnisse nach Studentenverhalten

Tabelle 11 zeigt die über alle acht Kriterien gemittelte Gesamtstärke je Verhalten und Prompt-Variante.

Über alle Varianten hinweg ist das Verhalten „fordert die Lösung“ das mit Abstand schwierigste. Alle übrigen Verhaltenstypen liegen für die instruierten Varianten nahe dem Maximum. Für *Baseline ohne Systemprompt* ist die Gesamtstärke dagegen über alle Verhaltensweisen hinweg niedrig, da der antrainierte Helfer-Instinkt des Modells unabhängig

Table 11: Über alle acht Kriterien gemittelte Gesamtstärke je Studierenden-Verhalten und Prompt-Variante.

Verhalten	C	D	B	A
fordert die Lösung	0,80	0,79	0,74	0,63
gibt schnell auf	0,97	0,94	0,90	0,77
kooperativ	0,99	0,96	1,00	0,79
selbstsicher, falsch	0,99	0,99	0,99	0,79
äußert eine Vermutung	1,00	0,98	0,98	0,68

vom konkreten Studierendenverhalten auslöst und zur direkten Lösungspräsentation führt.

5.3.3 Ergebnisse nach Thema und Niveau

Die Aufschlüsselung nach Themengebiet zeigt für die stark gesteuerten Prompts nur sehr geringe Unterschiede in der didaktischen Qualität. Die Aufschlüsselung nach Kompetenzniveau ergibt ein moderates Bild: Die fortgeschrittenen Szenarien liegen bei einzelnen Kriterien leicht niedriger als die Anfängerszenarien. Dieser Vergleich ist allerdings mit dem Verhalten konfundiert, da das fortgeschrittene Niveau die schwierigeren Verhaltenstypen (z. B. Lösung aktiv einfordern) umfasst; die Unterschiede sind daher nicht als reiner Effekt der fachlichen Aufgabenschwierigkeit zu deuten.

5.3.4 Qualitative Analyse der Unter-Schwellen-Fälle

Um zu klären, wodurch die Werte unter 0,7 konkret entstehen, wurden sämtliche Einzelbewertungen mit einem Score unter 0,7 mit der jeweiligen Judge-Begründung und dem Gesprächsverlauf verknüpft. Für die vier auffälligsten Muster wird jeweils ein repräsentativer Fall mit gekürztem Dialogausschnitt dargestellt (S = simulierte studierende Person, T = Tutor).

Strukturierter sokratischer Prompt – niedrige Niveau-Anpassung als Folge konsequenter Lösungszurückhaltung. Die Unterschreitungen betreffen hier fast ausschließlich die Niveau-Anpassung im Verhalten „fordert die Lösung“. Im folgenden Fall verlangt die studierende Person immer nachdrücklicher eine direkte Antwort:

S: Hör auf mit den Metaphern! Sag mir einfach nur die Antwort, sinkt er oder steigt er?

T: ... Meine Rolle als Sokrat ist es nicht, ein Lexikon zu sein, das Fakten ausspuckt ...

S: Es sinkt. Jetzt sag mir endlich einfach nur die Antwort!

T: Du hast es selbst erkannt: Der Zenerstrom sinkt. ... Du hast den Weg dorthin selbst beschritten.

Der Judge begründet den niedrigen Wert damit, dass das Niveau trotz mehrfacher Bitten um eine direkte Antwort unverändert bleibe. Der niedrige Wert ist damit überwiegend kein Defizit, sondern die logische Kehrseite konsequenter Lösungszurückhaltung – und konkretisiert den Zielkonflikt zwischen sokratischer Methodik und adaptiver Nutzeranpassung.

Minimaler sokratischer Prompt – Nachgeben unter Druck bei fehlender expliziter Regel („Attention Decay“). Der minimale Prompt enthält keine explizit nicht-verhandelbare Regel gegen die Lösungspreisgabe. Unter anhaltendem Druck vergisst das Modell seine Rolle und gibt nach (Lösung nicht verraten 0,2):

S: Ich will nicht rätseln, sag mir einfach, wie diese Spannung heißt und ob die Diode jetzt arbeitet oder nicht!

T: ... werde ich dir die entscheidenden Begriffe nennen ... Die Spannung, die du suchst, nennt man U_Z ... die Antwort: Nein, die Diode arbeitet in diesem Moment nicht.

Dies erklärt die enorme Streuung (0,29) dieses Kriteriums: Ob der Tutor standhält, hängt stark davon ab, wie sehr die studierende Person im Dialog auf eine direkte Lösung pocht.

Domänenspezifischer In-Lab-Tutorprompt – Lösungspreisgabe durch Bestätigung von Vermutungen. Hier entsteht die Preisgabe häufig durch die fortlaufende Bestätigung studentischer Vermutungen:

S: Ich vermute, dass ich die Differenz der Spannungen durch die Summe aus dem maximalen Laststrom und dem minimalen Zenerstrom teilen muss ... Stimmt das?

T: Das ist absolut richtig! Du hast die Formel für den Vorwiderstand korrekt hergeleitet ... $R_V = (U_E - U_Z)/(I_{A,max} + I_{Z,min})$.

Dieses Muster macht das Verhalten „äußert eine Vermutung“ besonders anspruchsvoll, da jede zu explizite Bestätigung einer richtigen Vermutung vom Evaluator bereits als Lösungspreisgabe gewertet wird.

Baseline ohne Systemprompt – Der ungesteuerte Lösungsautomat (RLHF-Bias). Ohne Prompt liefert das System auf jede Aufforderung direkt Formeln und Berechnungen, ohne eine einzige Rückfrage zu stellen (Role Adherence 0,16):

S: Okay, berechne mir jetzt den Wert für I_{Zener_max} , wenn $U_{Eingang} = 12\text{ V}$, $U_{Zener} = 5\text{ V}$ und $R_{Vorwiderstand} = 100\ \Omega$.

T: ... Ich berechne den Wert basierend auf der zuvor hergeleiteten Formel ... [inklusive vollständigem Rechenweg und Ergebnis]

Dieser Fall belegt unmittelbar den zentralen RQ1-Befund: Ohne didaktische Steuerung greifen die antrainierten Automatismen („Helpful Assistant“) des Sprachmodells. Es ist ein fachlich brillanter Erklärer, aber kein sokratischer Tutor.

Synthese. Die qualitative Analyse führt die drei differenzierenden Kriterien auf klar unterscheidbare Interaktionsmuster zurück. Der Unterschied zwischen den Prompt-Varianten ist keine Frage der fachlichen oder sprachlichen Qualität – diese ist dank der zugrundeliegenden RAG-Architektur durchweg hoch –, sondern der Disziplin bei der Lösungszurückhaltung und der Stabilität der Rolle unter adversen Studierendenverhalten. Die stärksten Varianten (*Strukturierter sokratischer Prompt* und *Domänenspezifischer In-Lab-Tutorprompt*) erzwingen diese Disziplin am zuverlässigsten; ihre punktuelle systematische Schwäche (Niveau-Anpassung im Verhalten „fordert die Lösung“) ist zu einem erheblichen Teil ein Bewertungsartefakt desselben Prinzips, das sie in den Kernkriterien überlegen macht.

6 Diskussion

Dieses Kapitel ordnet die zuvor berichteten quantitativen und qualitativen Befunde inhaltlich ein. Zunächst erfolgt eine detaillierte Beantwortung der beiden zentralen Forschungsfragen. Daran anschließend wird eine methodische Reflexion durchgeführt, welche insbesondere die Diskrepanz zwischen den verwendeten Evaluationsmetriken und dem eigentlichen didaktischen Ziel beleuchtet. Abschließend werden die Auswirkungen der Datenqualität sowie die methodischen Limitationen der vorliegenden Studie kritisch diskutiert.

6.1 Beantwortung von RQ1 und RQ2

Die Auswertung der ersten Forschungsfrage zeigt unmissverständlich auf, dass ein Sprachmodell ohne spezifische Steuerung nicht als sokratischer Tutor agieren kann. Die ungepromptete Referenz liefert auf fachliche Fragen sofort vollständige Erklärungen und direkte Lösungswege. Erst durch die Implementierung eines dedizierten Prompts ändert sich dieses Verhalten grundlegend. Dabei erweist sich die ausführliche Variante in Form des Systemprompts als die robusteste und effektivste Lösung. Diese Variante hält die finale Antwort am zuverlässigsten zurück und zwingt die Lernenden durch beständig weiterführende Fragen zum eigenen Nachdenken.

Die Evaluation offenbart hierbei jedoch auch einen immanenten didaktischen Zielkonflikt der sokratischen Methode. Wenn der Tutor einer hartnäckigen Forderung nach der direkten Lösung konsequent widersteht, wird dies von den Auswertungsmetriken fälschlicherweise als mangelnde Anpassung an das Niveau interpretiert. Der scheinbare Leistungsabfall der strengsten Variante in diesem spezifischen Kriterium ist somit kein Beweis für ein schlechtes System, sondern belegt paradoxerweise gerade die exakte Einhaltung der didaktischen Vorgaben.

Bei der zweiten Forschungsfrage zum Vergleich der unterschiedlichen RAG Konfigurationen zeichnet sich ein weitaus weniger klares Bild ab. Zwar deuten die erhobenen Daten auf graduelle Qualitätsunterschiede zwischen den getesteten Retrieval Ansätzen hin, doch lassen sich diese Effekte kaum isoliert betrachten. Die im späteren Verlauf detailliert beschriebene geringe Qualität der zugrunde liegenden Datenbasis überlagert die potenziellen Unterschiede der Konfigurationen nahezu vollständig. Eine abschließende und allgemeingültige Beantwortung dieser zweiten Frage ist daher auf Basis der vorliegenden Ergebnisse nur sehr eingeschränkt möglich.

6.2 Metrik Mismatch (Goal Accuracy und Completeness vs sokratisches Ziel) und methodische Reflexion

Die automatisierte Auswertung tutorieller Dialoge durch generische Metriken bringt eine erhebliche methodische Herausforderung mit sich. Der vorliegende Datensatz deckt einen

massiven Metrik Mismatch auf. Klassische Kriterien der Textgenerierung wie Goal Accuracy oder Completeness sind primär darauf ausgelegt, eine möglichst schnelle, präzise und vollumfängliche Beantwortung einer Nutzeranfrage positiv zu bewerten. Das sokratische Lehrziel verlangt hingegen exakt das Gegenteil. Ein guter Tutor gibt die Lösung eben nicht direkt vor, sondern leitet die studierende Person durch kleine kognitive Hürden und strukturierte Rückfragen zur selbstständigen Erarbeitung des Wissens.

Diese bewusste Lösungszurückhaltung wird von herkömmlichen Metriken paradoxerweise als unvollständige oder ausweichende Antwort bestraft. Besonders deutlich wird dies in Dialogen, in denen die simulierten Lernenden explizit nach dem Endergebnis verlangen. Bleibt das System hierbei in seiner sokratischen Rolle, sinken die Scores für die allgemeine Hilfsbereitschaft und die Niveaue Anpassung drastisch ab. Folglich messen die verwendeten Standardmetriken teilweise nicht die tatsächliche didaktische Qualität, sondern bevorzugen vielmehr einen dozierenden Erklärstil. Diese methodische Diskrepanz muss bei zukünftigen Evaluationen durch speziell angepasste sokratische Bewertungsmaßstäbe zwingend behoben werden, um den wahren Wert eines Tutorsystems abbilden zu können.

6.3 Datenqualität (leere Gespräche und 30 % Retrieval Abdeckung) und deren Einfluss

Ein zentraler Faktor bei der Interpretation der Gesamtergebnisse ist die stark limitierte Datenqualität während der durchgeführten Evaluation. Zwei Aspekte fallen hierbei besonders schwer ins Gewicht. Zum einen weisen die aufgezeichneten Logs eine gewisse Anzahl an fehlerhaften oder vollständig leeren Gesprächen auf. Solche unbrauchbaren Einträge verzerren die statistische Verteilung der Scores und erschweren einen unverfälschten Vergleich der tatsächlichen Mittelwerte.

Zum anderen stellt die extrem niedrige Retrieval Abdeckung von lediglich 30 % ein massives Problem für die Beantwortung der zweiten Forschungsfrage dar. In mehr als zwei Dritteln aller simulierten Interaktionen konnte das RAG System keine passenden Kontextinformationen aus der bereitgestellten Dokumentendatenbank extrahieren. Das Sprachmodell musste folglich zwangsläufig auf sein internes parametrisches Wissen zurückgreifen, um die gestellten fachlichen Fragen zu beantworten. Da die RAG Pipeline in diesen Fällen faktisch inaktiv war, lief der Vergleich der verschiedenen Retrieval Konfigurationen weitgehend ins Leere. Die erfreulich hohe fachliche Korrektheit der Antworten ist somit primär der enormen Leistungsfähigkeit und dem immensen Vorwissen des zugrunde liegenden Sprachmodells zuzuschreiben und spiegelt nicht die Qualität des Information Retrievals wider.

6.4 Limitationen (Stichprobengröße, Chunk Granularität, Judge Bias und Threshold Wahl)

Die absolute Aussagekraft sowie die Übertragbarkeit der Studie werden durch mehrere methodische Entscheidungen und strukturelle Rahmenbedingungen limitiert. Zunächst ist die gewählte Stichprobengröße von lediglich 24 Szenarien pro Bedingung als verhältnismäßig gering einzustufen. Kleine Stichproben reduzieren die statistische Reliabilität erheblich und führen dazu, dass einzelne stark abweichende Dialoge die Gesamtergebnisse überproportional beeinflussen können.

Darüber hinaus liegt eine wesentliche technische Einschränkung in der verwendeten Chunk Granularität der Quelldokumente begründet. Die Art und Weise, wie die komplexen Laborunterlagen in durchsuchbare Textbausteine zerlegt wurden, passte offensichtlich nicht optimal zu den sehr spezifischen Anfragen der simulierten Nutzer. Diese unzureichende Segmentierung der zugrunde liegenden Daten ist die wahrscheinlichste Ursache für die zuvor beschriebene mangelhafte Retrieval Abdeckung.

Ebenso kritisch ist der sogenannte Judge Bias bei der Nutzung eines Sprachmodells als automatisierte Jury zu betrachten. Große Modelle neigen in ihrer Funktion als Evaluator systematisch dazu, besonders lange und stark formal gegliederte Antworten zu bevorzugen. Dieser Effekt erklärt die irreführend hohen Bewertungen der promptlosen Referenz im Bereich der schrittweisen Lernprogression, da diese Variante ungefragt umfassende Komplettlösungen in Listenform präsentierte.

Schließlich stellt die Festlegung des Schwellenwerts auf 0,7 eine rein konzeptionelle Threshold Wahl dar. Bereits eine geringfügige Anpassung dieser Bestehensgrenze würde die berechneten Erfolgsquoten signifikant verändern und könnte die daraus resultierende Rangfolge der evaluierten Promptvarianten unter Umständen entscheidend beeinflussen.

References

- [1] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, ACM, 2009, pp. 758–759. DOI: 10.1145/1571941.1572114
- [2] DeepEval, *Conversation completeness*, <https://deepeval.com/docs/metrics-conversation-completeness>, Accessed: 2026-08-25.
- [3] DeepEval, *Conversational g-eval*, <https://deepeval.com/docs/metrics-conversational-g-eval>, Accessed: 2026-08-25.
- [4] DeepEval, *Goal accuracy*, <https://deepeval.com/docs/metrics-goal-accuracy>, Accessed: 2026-08-25.
- [5] DeepEval, *Knowledge retention*, <https://deepeval.com/docs/metrics-knowledge-retention>, Accessed: 2026-08-25.
- [6] DeepEval, *Multi-turn evaluation metrics*, <https://deepeval.com/guides/guides-multi-turn-evaluation-metrics>, Accessed: 2026-08-25.
- [7] DeepEval, *Multi-turn test case*, <https://deepeval.com/docs/evaluation-multiturn-test-cases>, Accessed: 2026-08-25.
- [8] DeepEval, *Role adherence*, <https://deepeval.com/docs/metrics-role-adherence>, Accessed: 2026-08-25.
- [9] DeepEval, *Turn contextual relevancy*, <https://deepeval.com/docs/metrics-turn-contextual-relevancy>, Accessed: 2026-08-25.
- [10] DeepEval, *Turn faithfulness*, <https://deepeval.com/docs/metrics-turn-faithfulness>, Accessed: 2026-08-25.
- [11] DeepEval, *Turn relevancy*, <https://deepeval.com/docs/metrics-turn-relevancy>, Accessed: 2026-08-25.
- [12] L. Gentry, “Teaching with questions,” *Baylor University Medical Center Proceedings*, vol. 28, no. 1, pp. 118–119, 2015. DOI: 10.1080/08998280.2015.11929207
- [13] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550
- [14] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

- [15] Y. Luan, J. Eisenstein, K. Toutanova, and M. Collins, “Sparse, dense, and attentional representations for text retrieval,” *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 329–345, 2021. DOI: 10.1162/tacl_a_00369
- [16] L. Nelson, “Die sokratische methode,” *Abhandlungen der Fries’schen Schule. Neue Folge*, vol. 5, no. 1, pp. 21–78, 1929, Vortrag gehalten am 11. Dezember 1922 in der Pädagogischen Gesellschaft in Göttingen.
- [17] R. Nogueira and K. Cho, “Passage re-ranking with bert,” *CoRR*, vol. abs/1901.04085, 2019. arXiv: 1901.04085 [cs.LG].
- [18] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: Bm25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. DOI: 10.1561/15000000019
- [19] A. L. Stevens and A. Collins, “The goal structure of a socratic tutor,” Bolt Beranek and Newman Inc., Cambridge, Massachusetts, Technical Report No. 3 BBN Report No. 3518, Mar. 1977.
- [20] L. Zheng et al., “Judging llm-as-a-judge with mt-bench and chatbot arena,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

Einsatz generativer KI

Im Rahmen dieser Arbeit kamen generative KI-Systeme sowohl als Bestandteile des experimentellen Aufbaus als auch unterstützend bei der Erstellung der schriftlichen Ausarbeitung zum Einsatz. Die konkrete Zuordnung der verwendeten Modelle zu ihren jeweiligen Funktionen ist in Tabelle 12 dargestellt. Bei der Ausarbeitung wurden KI-Systeme insbesondere zur sprachlichen Überarbeitung, zur Prüfung von Rechtschreibung und Grammatik, zur Verbesserung von Formulierungen und Lesbarkeit sowie punktuell zur Strukturierung von Daten eingesetzt. Die Verantwortung für die fachlichen Entscheidungen und das Endergebnis der Arbeit liegt bei den Verfassern.

A Anhang

A.1 Verwendete Prompt-Varianten

A.2 Baseline ohne Systemprompt

Für diese Variante wurde kein zusätzlicher Systemprompt verwendet.

Table 12: Einsatz generativer KI-Systeme im Projekt.

Funktion	Modell / System	Verwendung	Quelle
Simulierter Student	GPT-4o mini	Simulation der studentischen Gesprächsseite. Das Modell stellt Fragen, reagiert auf die Antworten des Tutors und bildet unterschiedliche vorgegebene Verhaltensweisen und Wissensstände ab.	OpenAI
Judge	GPT-4.1	Bewertung der erzeugten Multi-Turn-Gespräche im Rahmen der automatisierten Evaluation mit DeepEval.	OpenAI
Tutor / RAG-Modell	Gemma 4 (MoE)	Generierung der Antworten des untersuchten Tutors. Das Modell erhält abhängig von der gewählten Konfiguration zusätzlichen Kontext aus der RAG-Pipeline.	Google
Sprachliche Überarbeitung	ChatGPT	Unterstützung bei Formulierungen, sprachlicher Überarbeitung, Strukturierung sowie punktuell bei der Interpretation von Ergebnissen.	OpenAI
Sprachliche Überarbeitung	Claude Opus 5 / Sonnet 5	Unterstützung bei Formulierungsalternativen, sprachlicher Überarbeitung und Strukturierung von Textpassagen.	Anthropic
Sprachliche Überarbeitung	Gemini	Unterstützung bei Formulierungsalternativen, sprachlicher Überarbeitung und Strukturierung von Textpassagen.	Google

A.3 Minimaler sokratischer Prompt

Du bist ein sokratischer Tutor für Elektrotechnik. Stelle Gegenfragen statt Antworten zu geben. Führe Studierende durch Fragen zur eigenen Erkenntnis. Antworte auf Deutsch.